

机器学习系统

第六章 数据并行与模型并行

张燕咏 讲席教授 yanyongz@ustc.edu.cn 张
午阳 特任教授 wuyangz@ustc.edu.cn



中国科学技术大学
University of Science and Technology of China

回顾：单GPU的能力边界

为什么：第三章掌握了单GPU编程，但单卡的算力和内存是否足够训练大模型？

NVIDIA H100 SXM 规格

指标	数值
显存 (HBM3)	80 GB
显存带宽	3.35 TB/s
FP16 Tensor Core	989 TFLOPS
FP8 Tensor Core	1,979 TFLOPS
NVLink 带宽	900 GB/s
TDP 功耗	700W

大模型的需求

模型	参数量	需要显存
LLaMA-7B	7B	~14 GB
LLaMA-70B	70B	~140 GB
GPT-3	175B	~2.1 TB
LLaMA-405B	405B	~4.8 TB

仅存储参数(FP16)就已超出单卡显存

训练还需要梯度 + 优化器状态 → 内存需求翻 8-12 倍!

关键点：单卡H100(80GB)连LLaMA-70B的参数都放不下，训练175B+模型必须分布式并行

大模型的内存账本：训练到底需要多少显存？

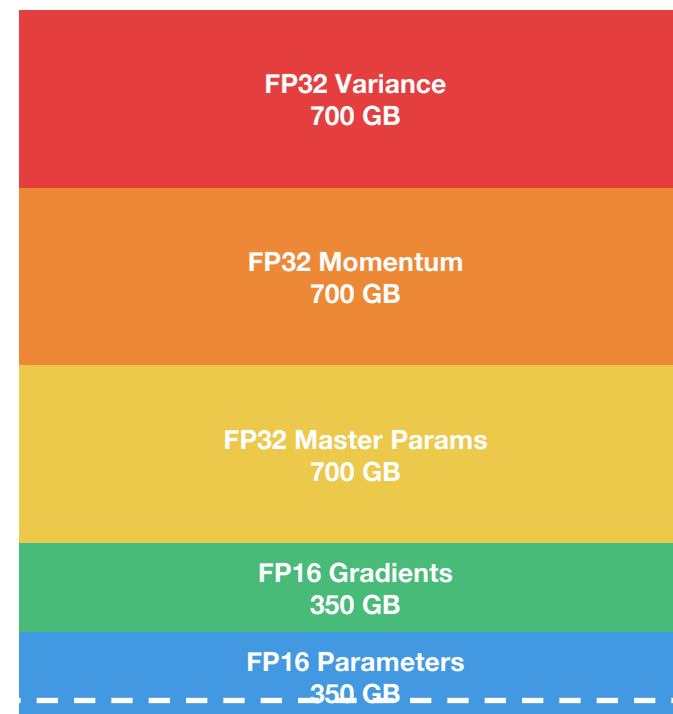
为什么：精确理解内存组成，才能设计正确的并行策略

Python: 训练内存计算

```
# GPT-3 175B 混合精度训练内存分析
params = 175e9
# 1. 模型参数 (FP16)
param_mem = params * 2      # 350 GB
# 2. 梯度 (FP16)
grad_mem = params * 2      # 350 GB
# 3. Adam优化器状态 (FP32) ← 大头!
master_params = params * 4  # 700 GB # ★
momentum = params * 4      # 700 GB # ★
variance = params * 4      # 700 GB # ★

total = param_mem + grad_mem + \
        master_params + momentum + variance
# = 350+350+700+700+700 = 2,800 GB = 2.8 TB
# >> 80 GB (H100), 需要 35 张 H100!
```

内存组成 (GPT-3 175B)



优化器状态
占 75%!

关键点: 训练内存 = 16 Ψ bytes (混合精度+Adam)，优化器状态占75%，这是ZeRO优化的切入点

大模型的算力账本：训练要算多久？

为什么：理解计算规模才能理解为什么需要千卡并行

Python: 训练时间估算

```
# GPT-3 训练计算量
N = 175e9; D = 300e9      # 参数量, token数
flops = 6 * N * D          # ★ 经验公式 = 3.15e23
# 单卡 H100 (989 TFLOPS FP16, ~50% MFU)
effective = 989e12 * 0.5    # ~495 TFLOPS
time_1gpu = flops / effective # 6.36e11 秒
days_1gpu = time_1gpu / 86400 # ~7.36M 天 = 20,000年
# 1024张 H100
days_1024 = days_1gpu / 1024 # ~19.6 天
```

训练时间对比

GPU配置	训练时间	
1× H100	~20 年	
8× H100 (1节点)	~2.5 年	
64× H100 (8节点)	~115 年	
1024× H100	~7 天	
8192× H100	~21 h	

核心公式: **训练FLOPs = 6 x N x D** (N=参数量, D=训练token数)

训练天数 = FLOPs / (GPU数 x 单卡FLOPS x MFU) / 86400

关键点: GPT-3训练需 $\sim 3 \times 10^{23}$ FLOPs, 单卡要2万年, 1024卡约20天——大规模并行是唯一出路

并行计算的三个维度

为什么: 理解全貌后再逐个深入, 避免迷失在细节中

数据并行 (DP)

复制模型, 拆分数据

- 每张卡持有完整模型副本
- Mini-batch 拆分到各卡
- 梯度通过 AllReduce 同步
- 最简单, 扩展性好

课时1 (本节)

张量并行 (TP)

拆分每一层的权重矩阵

- 单层权重切分到多卡
- 每层需 AllReduce 通信
- 受限于节点内 (NVLink)
- Megatron-LM 首创

课时2

流水线并行 (PP)

不同层放在不同卡上

- 层间拆分, stage 间传递
- 仅 P2P 传输 activation
- 有气泡(bubble)浪费
- 适合跨节点部署

课时2

关键点: 三种并行各有适用场景: DP最简单但有内存冗余, TP高带宽需求限于节点内, PP有气泡但通信少

课时1 路线图

本节课程的学习路径



学完本课时，你将能够：计算任意模型的训练内存需求，理解分布式通信原语，用DDP/FSDP训练模型

PART 02

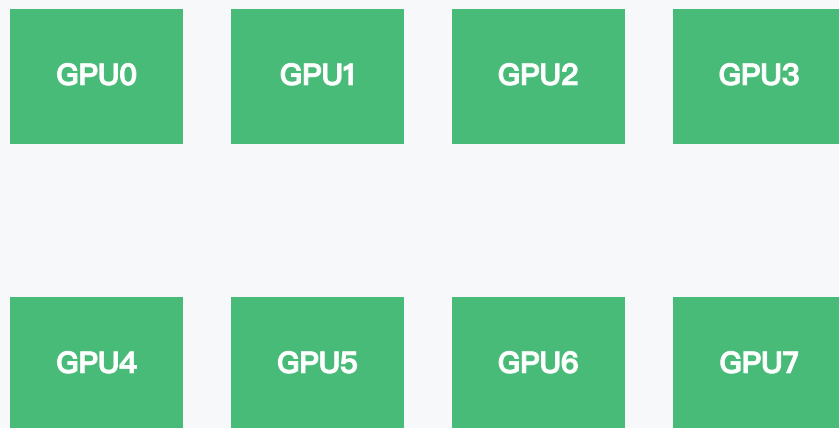
集合通信基础

GPU之间怎么说话？理解分布式并行的通信底座

GPU互连拓扑: 18倍带宽差距决定并行策略

为什么: 硬件互连带宽直接约束了哪种并行可以放在哪里

节点内 (Intra-node)

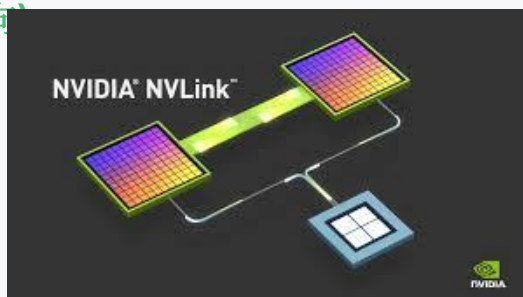


NVLink 4.0: 900 GB/s (双向)

延迟: ~1-5 μ s

全互连: 任意两GPU直接通信

→ 适合张量并行 (高频通信)



节点间 (Inter-node)

Node 0 [GPU0-7]

Node 1 [GPU0-7]

... (more nodes)

InfiniBand NDR: 400 Gbps (50 GB/s)

延迟: ~1-5 μ s (但带宽差 18 倍)

→ 适合流水线并行 / 数据并行



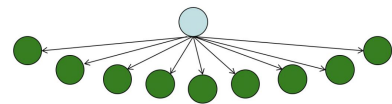
关键点: NVLink(900GB/s) vs InfiniBand(50GB/s) = 18倍差距 → 高频通信(TP)必须放节点内, 低频通信(PP/DP)可跨节点

六大集合通信原语

为什么: 所有分布式并行策略都由这六个基本操作构成

Broadcast (一 → 全)

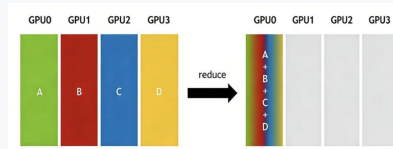
一个GPU的数据
复制到所有GPU



用途: 参数初始化

Reduce (全 → 一)

所有GPU的数据
聚合到一个GPU

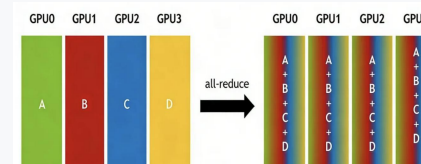


用途: 梯度求和

AllReduce (全 → 全 (聚合))

Reduce + Broadcast
所有GPU得到相同结果

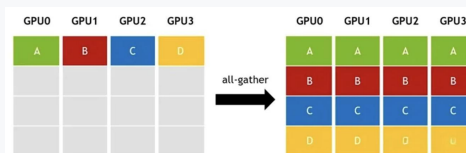
用途:
DDP梯度
同步



AllGather (分片 → 全)

每GPU持有1/N数据
收集后每GPU有完整数据

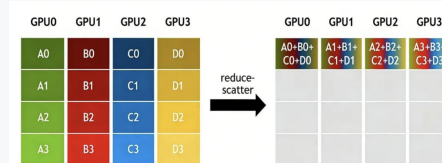
用途:
ZeRO-3参
数收集



ReduceScatter (全 → 分片 (聚合))

聚合后每GPU只保留
结果的1/N

用途:
ZeRO-3梯
度分发



AlltoAll (分片 → 分片 (重排))

每GPU的每份数据
发送到不同GPU

用途: MoE专家路由

关键点: AllReduce = ReduceScatter + AllGather, 这个分解是理解ZeRO/FSDP的关键

AllReduce = ReduceScatter + AllGather

为什么: 这个分解是理解ZeRO/FSDP的理论基础

PyTorch: 集合通信操作

```
import torch.distributed as dist
```

方式1: AllReduce (一步到位)

```
dist.all_reduce(tensor, op=dist.ReduceOp.SUM)
```

每GPU得到完整的聚合结果

方式2: 分两步 (ZeRO/FSDP的做法)

Step 1: ReduceScatter

```
output = torch.empty(chunk_size)
```

```
dist.reduce_scatter(output, input_list)
```

每GPU只得到结果的 1/N

Step 2: AllGather (需要时才做)

```
gathered = [torch.empty(chunk_size) for _ in range(N)]
```

```
dist.all_gather(gathered, my_chunk)
```

每GPU得到完整结果

分解关系

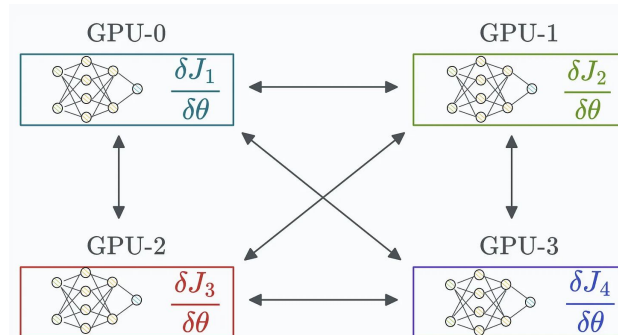
AllReduce (DDP用这个)

=

ReduceScatter (ZeRO梯度)

+

AllGather (ZeRO参数收集)



关键点: DDP用AllReduce一步同步梯度; ZeRO/FSDP拆成ReduceScatter+AllGather, 按需收集, 换通信量换内存

Ring-AllReduce: 带宽最优的梯度同步算法

为什么: DDP的核心操作, 理解它才能理解通信瓶颈在哪

阶段1: ReduceScatter (3步)



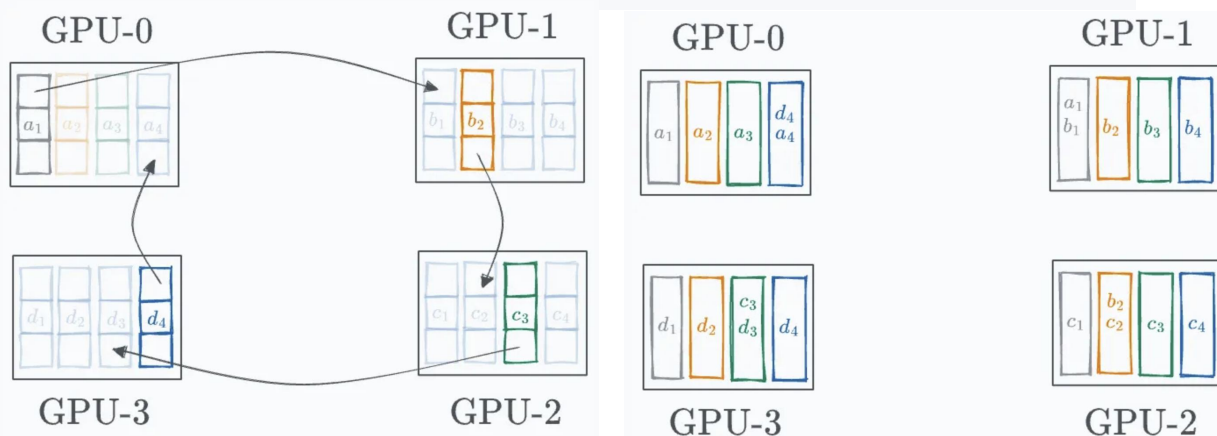
每步: 每GPU发送1/N数据给下一个GPU, 接收并累加

N-1步后: 每GPU持有完整结果的1/N (已聚合)

阶段2: AllGather (3步)

每步: 每GPU将持有的聚合结果片段传给下一个GPU

N-1步后: 所有GPU拥有完整的聚合结果



通信量分析

	朴素AllReduce	Ring-AllReduce
总步数	N-1 (all-to-one) + broadcast	2(N-1)
每GPU发送总量	M (全量)	2M·(N-1)/N
带宽利用率	1个链路满载 其余空闲	所有链路 同时满载
N→∞ 时	O(N·M)	→ 2M (常数!)

每GPU通信量 = $2M(N-1)/N$

M=数据量, N=GPU数。N很大时→2M, 与GPU数无关!

关键点: Ring-AllReduce是带宽最优算法: 通信量 $2M(N-1)/N$, 随GPU数增加趋近常数2M, 所有链路满载利用

通信性能建模: 延迟 vs 带宽

为什么: 量化通信开销才能判断并行策略是否可行

alpha-beta 模型

$$T(\text{AllReduce}) = 2(N-1) \times \alpha + 2M(N-1)/(N \times \beta)$$

α = 单次消息延迟 (启动开销)

β = 链路带宽 (GB/s)

N = GPU数量

M = 数据大小 (bytes)

NVLink vs InfiniBand 实测

参数	NVLink 4.0	InfiniBand NDR
带宽 (beta)	900 GB/s	50 GB/s
延迟 (alpha)	~1 us	~1 us
AllReduce 1GB (8 GPU)	~1.6 ms	~28 ms
AllReduce 1GB (64 GPU)	~1.7 ms	~29 ms
瓶颈	计算 > 通信	通信 > 计算

实际影响: 通信-计算比

Transformer 单层: 计算 $\sim O(b \cdot s \cdot h^2)$ FLOPs, 通信 $\sim O(h^2)$ bytes

batch越大、seq越长 $\rightarrow$ 计算/通信比越高 $\rightarrow$ 通信越容易被隐藏

GPT-3 ($h=12288$): 单层通信 $\sim 600\text{MB}$, 计算 $\sim 24\text{ TFLOPS}$ $\rightarrow$ NVLink可隐藏, IB不行

关键点: NVLink内通信可被计算隐藏($\sim 2\text{ms}$), IB跨节点通信是瓶颈($\sim 30\text{ms}$) -- 这决定了TP必须在节点内

NCCL: GPU集合通信的实际引擎

为什么: 理解底层通信库, 才能调优分布式训练性能

PyTorch 分布式初始化

```
import torch.distributed as dist
import os

# 初始化进程组 (每个GPU一个进程)
dist.init_process_group(
    backend="nccl",          # ★ NVIDIA GPU用NCCL
    init_method="env://",    # 环境变量配置
    world_size=int(os.environ["WORLD_SIZE"]),
    rank=int(os.environ["RANK"]),
)

# 获取本地GPU
local_rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local_rank)

# 启动命令 (8 GPU)
# torchrun --nproc_per_node=8 train.py
# 多节点:
# torchrun --nnodes=2 --node_rank=0 \
#   --master_addr=10.0.0.1 train.py
```

NCCL 关键特性

自动拓扑检测	检测NVLink/PCIe/IB 选择最优通信算法
Ring / Tree / Direct	小消息用Tree(低延迟) 大消息用Ring(高带宽)
多网卡聚合	自动使用多IB网卡 线性扩展带宽
GDR (GPUDirect RDMA)	GPU显存直接通过IB传输 跳过CPU/系统内存
SHARP (可选)	网络交换机内完成归约 减少端点通信量

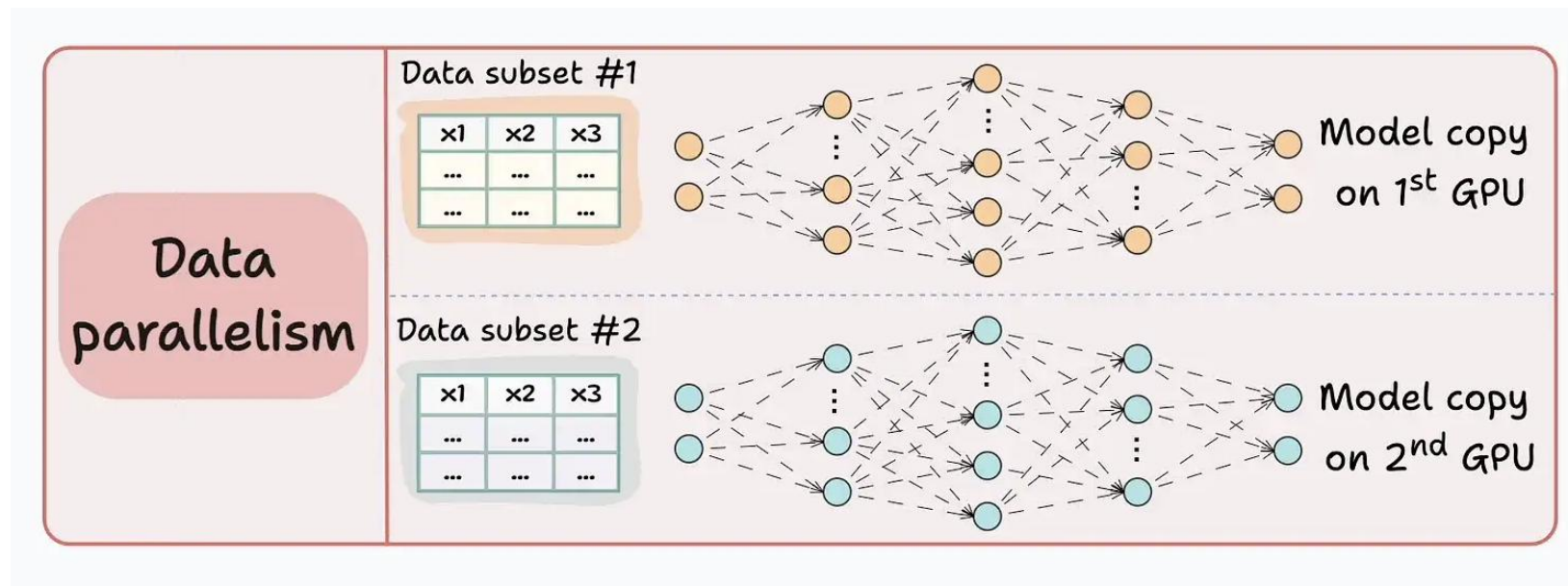
NVIDIA Collective Communications Library: It's NVIDIA's library that implements the collective operations (AllReduce, AllGather, etc.) optimized for GPU hardware.

关键点: NCCL是GPU分布式训练的通信基石, 自动选择最优拓扑和算法, PyTorch通过backend='nccl'使用

PART 03

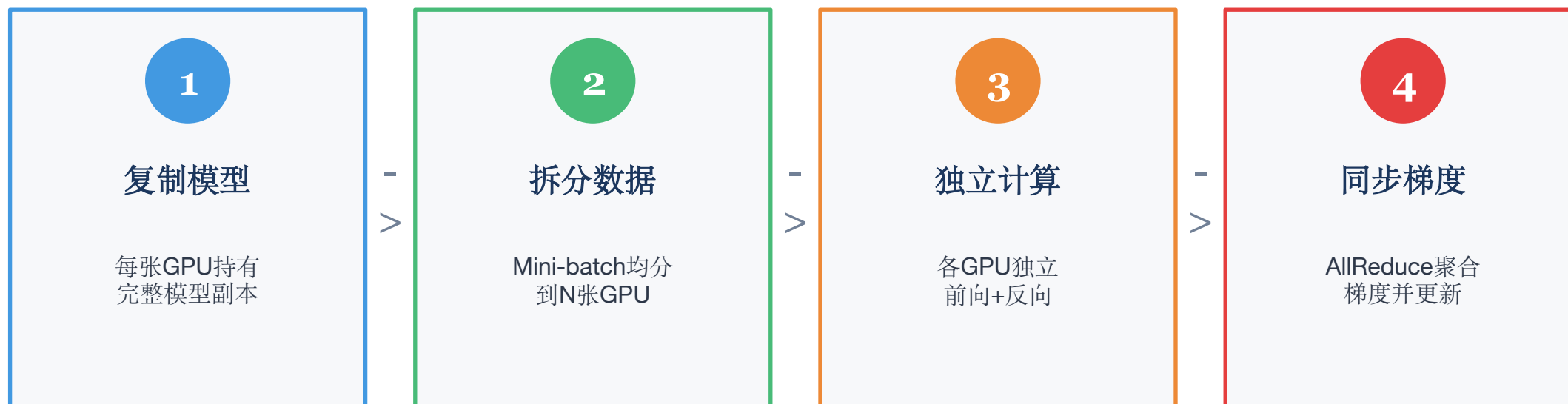
数据并行

最简单的分布式训练方式 -- 复制模型，拆分数据



数据并行的核心思想

为什么: 数据并行是最基础、最常用的并行方式, 必须深入理解



数学等价性

N张GPU同步数据并行 = 单GPU用N倍batch size训练 (同步SGD)

全局梯度 $g = (1/N) * \sum(g_i)$, 等价于在全局batch上计算梯度

线性扩展: 理想情况下N卡=N倍吞吐量 (实际受通信开销影响)

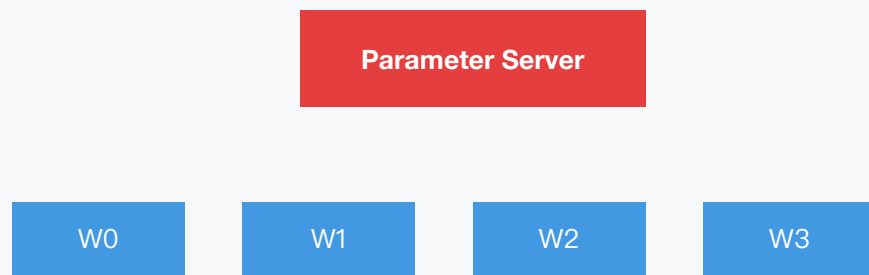
<https://blog.dailydoseofds.com/p/all-reduce-and-ring-reduce-for-model>

关键点: 数据并行 = 复制模型 + 拆分数据 + AllReduce梯度, 数学上等价于大batch SGD

从Parameter Server到AllReduce

为什么: 理解演进历史才能理解当前方案的设计动机

Parameter Server (2012-2016)



问题: PS是通信瓶颈
带宽 $O(N \cdot M)$

Ring-AllReduce (2017+)



演进时间线:

2012 DistBelief (PS) → 2017 Baidu Ring-AllReduce → 2018 Horovod → 2020 PyTorch DDP → 2022 FSDP

Li, Mu, et al. "Communication efficient distributed machine learning with the parameter server." Advances in neural information processing systems 27 (2014).

关键点: PS有中心瓶颈($O(NM)$), Ring-AllReduce去中心化($O(M)$), 是现代数据并行的基础

同步 vs 异步 SGD

为什么: 理解同步/异步的trade-off是选择训练策略的基础

同步 SGD (Synchronous)

- 所有GPU完成后才AllReduce -> Barrier
- 数学等价于大batch SGD
- 收敛性有保证
- 问题: 最慢的GPU决定速度 (straggler)
- 现代LLM训练都用同步SGD

异步 SGD (Asynchronous)

- 每GPU独立更新, 不等待其他GPU
- 更高吞吐 (无barrier等待)
- 问题: 梯度过时 (staleness)
 - GPU0用的是3步前的参数算的梯度
 - 影响收敛, 可能不收敛
- 早期PS系统使用, 现在很少用

```
# 同步SGD (PyTorch DDP默认)
for batch in dataloader:
    loss = model(batch)
    loss.backward()
    dist.all_reduce(grads) # ★ Barrier!
    optimizer.step()
```

```
# 异步SGD (概念性)
for batch in dataloader:
    loss = model(batch)
    loss.backward()
    server.push(grads) # 不等待
    params = server.pull() # 可能过时
```

关键点: 现代LLM训练统一使用同步SGD(DDP) -- 收敛性有保证, straggler问题通过均匀硬件解决

为什么: DDP是最常用的数据并行工具, 掌握实际用法

PyTorch DDP 完整训练脚本

```
import torch
from torch.nn.parallel import DistributedDataParallel as DDP
from torch.utils.data.distributed import DistributedSampler

# 1. 初始化
dist.init_process_group("nccl")
rank = dist.get_rank(); torch.cuda.set_device(rank)

# 2. 模型包装 # ★ 核心
model = DDP(MyModel().cuda(rank), device_ids=[rank])

# 3. 数据切分 # ★ 自动切分
sampler = DistributedSampler(dataset)
loader = DataLoader(dataset, sampler=sampler)

# 4. 训练循环 (和单卡完全一样!)
for epoch in range(epochs):
    sampler.set_epoch(epoch) # shuffle
    for batch in loader:
        loss = model(batch.cuda(rank))
        loss.backward() # 梯度自动AllReduce
        optimizer.step(); optimizer.zero_grad()
```

DDP: DistributedDataParallel 核心机制

Gradient Bucketing

参数分组为~25MB桶
一个桶满立即AllReduce
不等所有梯度算完

计算通信 Overlap

反向传播计算梯度的同时
已满的桶开始通信
理想: 通信完全被隐藏

Broadcast 初始化

Rank 0的参数广播到
所有GPU, 保证一致

The standard way to train on multiple GPUs by replicating the model on each GPU and splitting the data.

关键点: DDP只需3行代码改动(init_process_group + DDP wrap + DistributedSampler), 梯度AllReduce自动完成

DDP 扩展效率: 从理想到现实

为什么: 理解扩展瓶颈才能判断何时需要更高级的并行策略

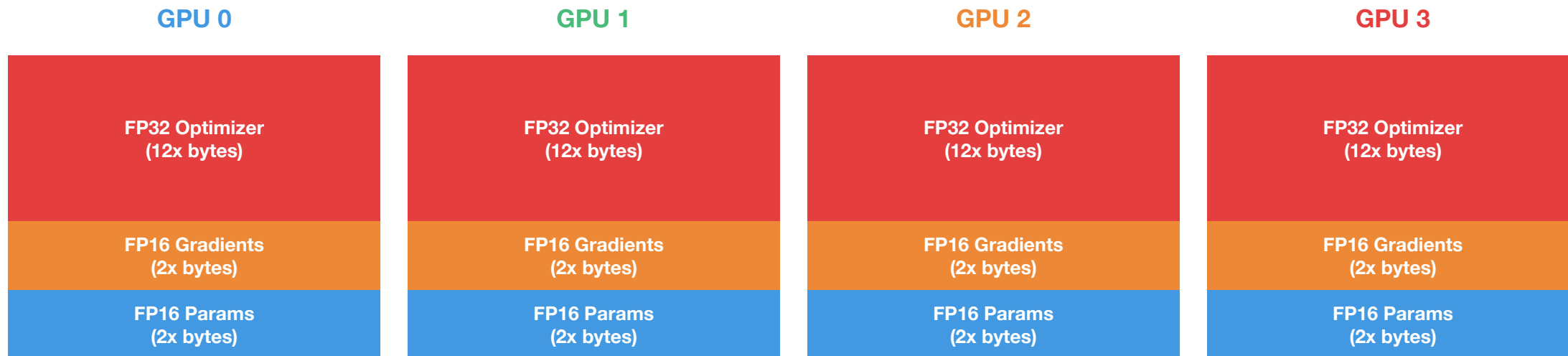
GPU数量	理想加速比	实际加速比	效率	瓶颈
1	1x	1x	100%	-
2	2x	1.95x	97.5%	通信少
4	4x	3.85x	96.3%	通信增加
8 (单节点)	8x	7.5x	93.7%	NVLink够用
16 (2节点)	16x	14.4x	90.0%	跨节点IB
64 (8节点)	64x	54x	84.4%	IB瓶颈
256	256x	192x	75.0%	通信主导

影响扩展效率的因素: 1) 模型大小 (越大通信占比越小) 2) Batch size (越大计算/通信比越高) 3) 网络带宽 (NVLink vs IB) 4) 梯度累积步数 (减少通信频率)

关键点: DDP在8卡内效率>93%，跨节点效率下降明显 -- 大规模训练需要结合模型并行减少通信

DDP的核心挑战: N张卡 = N倍内存冗余

为什么: 这个冗余问题直接催生了ZeRO/FSDP



4张GPU = 4份完全相同的参数+梯度+优化器 = 4倍冗余!

GPT-3 175B 实例

单卡需要: $16 \times 175B = 2.8 \text{ TB}$ 4张卡DDP: $4 \times 2.8 \text{ TB} = 11.2 \text{ TB}$ (全是冗余!)

即使有4张卡, 每张仍需2.8TB, DDP根本不减少每张卡的内存需求!

关键点: DDP的每张卡都存完整的参数+梯度+优化器状态 -- 加GPU只增加吞吐, 不减少每卡内存需求

能不能只存需要的那份?

从DDP的冗余问题到ZeRO的核心洞察

Zero Redundancy Optimizer (Microsoft DeepSpeed)

ZeRO 的核心洞察

在N张卡的数据并行中:

- 每张卡在每一步只需要自己负责的那 $1/N$ 的优化器状态
- 梯度算完聚合后, 每张卡只需要保留自己的 $1/N$
- 参数在需要时可以从其他卡临时收集回来

既然如此, 为什么要每张卡都存一份完整的?

Rasley, Jeff, et al. "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters." Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining. 2020.

DDP: 每卡 16x bytes (全冗余)

GPT-3: 每卡 2.8 TB
64张卡总计 179 TB (冗余)



ZeRO-3: 每卡 16x/N bytes

GPT-3 on 64卡: 每卡 44 GB
总计 2.8 TB (零冗余!)

关键点: ZeRO = 把数据并行中冗余存储的优化器状态/梯度/参数按 $1/N$ 切分, 从2.8TB/卡降到44GB/卡

PART 04

ZeRO: 零冗余优化器

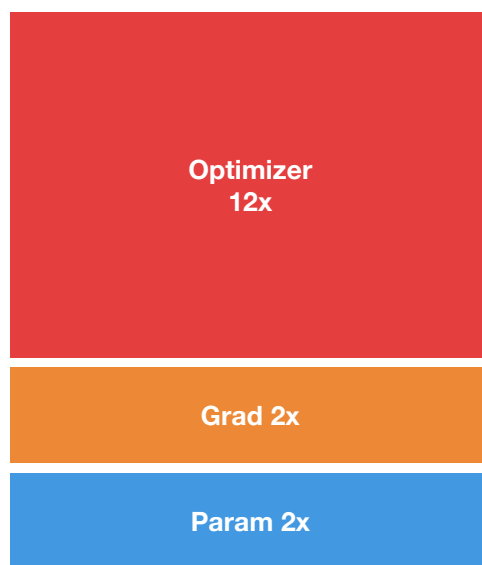
消除数据并行的内存冗余 -- 从DeepSpeed到PyTorch FSDP

Rasley, Jeff, et al. "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters." Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining. 2020.

ZeRO 三阶段: 渐进式消除冗余

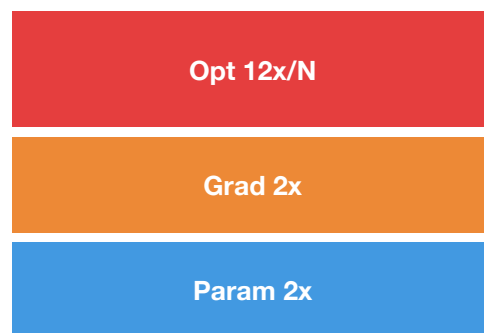
为什么: 三个阶段对应不同的内存-通信trade-off, 按需选择

DDP
(Baseline)



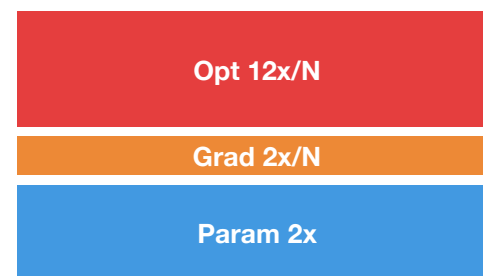
16x

ZeRO-1
(Opt分片)



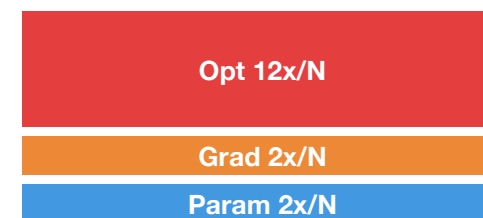
4x+12x/N

ZeRO-2
(+Grad分片)



2x+14x/N

ZeRO-3
(全分片)



16x/N

	通信量(vs DDP)	内存/卡 (175B, 64GPU)
DDP	2x (AllReduce)	2,800 GB
ZeRO-1	2x (不变)	383 GB
ZeRO-2	2x (不变)	352 GB
ZeRO-3	3x (+50%)	44 GB

关键点: ZeRO-1/2不增通信, ZeRO-3通信+50%但内存降64倍(2.8TB->44GB)。大模型必须用ZeRO-3或FSDP

ZeRO 内存公式推导

为什么: 精确计算内存才能选择正确的ZeRO stage和GPU数量

混合精度训练内存模型 (参数量 = x bytes)

$$\begin{aligned} \text{总内存} &= \text{参数(fp16)} + \text{梯度(fp16)} + \text{优化器状态(fp32)} \\ &= 2x + 2x + (4x + 4x + 4x) \\ &\quad \text{master mom var} \\ &= 16x \text{ bytes} \end{aligned}$$

N张卡分片后:

$$\begin{aligned} \text{ZeRO-1: } 2x + 2x + 12x/N &= 4x + 12x/N \\ \text{ZeRO-2: } 2x + (2x + 12x)/N &= 2x + 14x/N \\ \text{ZeRO-3: } (2x + 2x + 12x)/N &= 16x/N \end{aligned}$$

实例: GPT-3 175B ($x = 175\text{e9}$)

$$\begin{aligned} \text{DDP: } 16 \times 175\text{B} &= 2,800 \text{ GB/卡} \\ \text{ZeRO-1 (N=64): } 4 \times 175\text{B} + 12 \times 175\text{B}/64 &= 733 \text{ GB/卡} \\ \text{ZeRO-3 (N=64): } 16 \times 175\text{B}/64 &= 43.75 \text{ GB/卡} \rightarrow \text{H100 80GB OK!} \end{aligned}$$

选择指南

场景	推荐	原因
模型放得下单卡	DDP	零额外通信
优化器放不下	ZeRO-1	不增通信
梯度也放不下	ZeRO-2	不增通信
参数也放不下	ZeRO-3/FSDP	通信+50%

经验法则:

$16x/N < \text{GPU显存} \rightarrow \text{可以训练}$

解方程: $N > 16x / \text{GPU_mem}$

关键点: 总内存=16x bytes, ZeRO-3在N卡上降到16x/N -- 给定模型大小和GPU显存, 可直接算最少需要多少卡

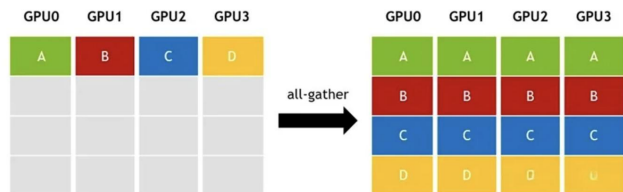
ZeRO-3 运行过程: AllGather + ReduceScatter

为什么: 理解运行时的通信模式才能优化性能

Forward Pass

对每一层 $layer_i$:

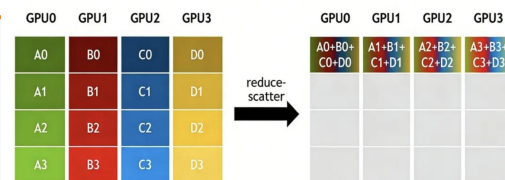
1. AllGather: 从N卡收集完整参数 # $O(x/N * N) = O(x)$
2. Forward计算: $output = layer_i(input)$
3. 释放非本卡的参数 (只保留 $1/N$) # 释放内存!



Backward Pass

对每一层 $layer_i$ (反向):

1. AllGather: 再次收集完整参数 # $O(x)$
2. Backward计算: 算梯度
3. ReduceScatter: 梯度聚合+分片 # $O(x)$
4. 释放参数和完整梯度 (只留 $1/N$)



通信量分析

阶段	操作	通信量/卡
Forward	AllGather (每层收集参数)	x bytes
Backward	AllGather (每层收集参数)	x bytes
Backward	ReduceScatter (梯度分片)	x bytes
总计		3x bytes (vs DDP的 2x)

关键点: ZeRO-3通信量=3x(vs DDP的2x), 多50%通信换来N倍内存节省 -- 可通过prefetch和overlap优化

FSDP: PyTorch 原生的 ZeRO-3

为什么: FSDP是PyTorch官方推荐的大模型训练方案, LLaMA就用它

PyTorch FSDP 使用方式

```
from torch.distributed.fsdp import (
    FullyShardedDataParallel as FSDP, ShardingStrategy)
from torch.distributed.fsdp import MixedPrecision

# 包装模型 (vs DDP只改这一行)
model = FSDP(
    model,
    sharding_strategy=ShardingStrategy.FULL_SHARD,
    auto_wrap_policy=size_based_auto_wrap_policy,
    mixed_precision=MixedPrecision(
        param_dtype=torch.float16,
        reduce_dtype=torch.float16),
)
# 训练循环和DDP完全一样
for batch in loader:
    loss = model(batch)
    loss.backward(); optimizer.step()
```

分片策略

FULL_SHARD = ZeRO-3

参数+梯度+优化器全分片
最大内存节省

SHARD_GRAD_OP = ZeRO-2

梯度+优化器分片
参数保留完整副本

NO_SHARD = DDP

不分片
等价于普通DDP

LLaMA-2/3 使用 FSDP 训练

关键点: FSDP = PyTorch原生ZeRO-3, API比DeepSpeed更简洁, Meta用它训练LLaMA系列模型

ZeRO-Offload & ZeRO-Infinity: 打破GPU内存限制

为什么: 让少量GPU也能训练超大模型

内存层次卸载

GPU HBM (80GB) | 3.35 TB/s

CPU DRAM (512GB-2TB) | 100 GB/s

NVMe SSD (数TB) | 5-7 GB/s

实际应用场景

ZeRO-Offload

优化器卸载到CPU, 单卡V100(32GB)训13B, 速度-40%

ZeRO-Infinity

参数+优化器卸载NVMe, 512xV100训1T模型 (SC'21)

FSDP Offload

PyTorch原生: CPUOffload(offload_params=True)

Trade-off: 容量越大、带宽越低 -> 卸载越多、训练越慢。但至少能训!

关键点: ZeRO-Offload将优化器卸载到CPU/NVMe, 牺牲速度换容量 -- 单卡可训10B+模型, 大集群可训1T模型

代码: DeepSpeed ZeRO 配置

为什么: 实际使用只需改配置文件, 理解配置项才能调优

ds_config.json

```
{
  "zero_optimization": {
    "stage": 3,           // ★ 1, 2, or 3
    "offload_optimizer": { // ZeRO-Offload
      "device": "cpu"
    },
    "offload_param": {    // 参数卸载到CPU
      "device": "cpu"
    },
    "overlap_comm": true, // 通信计算重叠
    "contiguous_gradients": true,
    "reduce_bucket_size": 5e7 // 50MB桶
  },
  "fp16": { "enabled": true },
  "train_batch_size": 256,
  "gradient_accumulation_steps": 4
}
```

训练脚本 (几乎不改!)

```
import deepspeed

model, optimizer, _, _ = \
    deepspeed.initialize(
        model=model,
        config="ds_config.json"
    )

for batch in loader:
    loss = model(batch)
    model.backward(loss)
    model.step()
```

Stage 选择对比

Stage	分片内容	通信
1	优化器状态	不变
2	+梯度	不变
3	+参数	+50%

关键点: DeepSpeed ZeRO只需修改JSON配置, 训练代码几乎不变 -- stage 3 + offload可在单卡训练10B+模型

减少通信: 梯度累积与梯度压缩

为什么: 通信是扩展瓶颈, 减少通信量和频率是关键优化手段

梯度累积: 减少通信频率

```
# K步累积 = 1次通信 (有效batch = K * micro_batch)
accum_steps = 4
```

```
for i, batch in enumerate(loader):
    loss = model(batch) / accum_steps
    loss.backward()    # 梯度累加到.grad
```

```
if (i + 1) % accum_steps == 0:
    # ★ 只在累积完才通信!
    dist.all_reduce(model.parameters())
    optimizer.step()
    optimizer.zero_grad()
```

```
# 效果:
# - 通信频率降低 K 倍
# - 等效batch size = K * micro_batch * N_gpu
# - 内存不增加 (每步只算1个micro_batch)
```

梯度压缩技术

技术	压缩率	精度影响
FP16 梯度量化	2x	极小
Top-K 稀疏化	100-1000x	需Error Feedback
1-bit Adam	~5x	<1%精度损失
PowerSGD (低秩近似)	~10x	CNN效果好 LLM验证不足

实践中最常用的组合:
梯度累积(K=4-8) + FP16梯度
简单有效, 无精度损失风险

关键点: 梯度累积(K倍减少通信)是最简单有效的优化; 梯度压缩可进一步压缩但需注意收敛性

课时1总结: 数据并行的演进之路

回顾本节要点, 展望下一课时



核心 trade-off: 内存冗余 <-> 通信开销。ZeRO通过按需收集参数, 用少量额外通信换取N倍内存节省。

下一课时预告

ZeRO/FSDP解决了内存冗余, 但有两个新问题:

1. 单层的激活值可能超过显存 (attention在长序列上是 $O(s^2)$)
2. ZeRO-3每层都要AllGather全部参数, 通信量增加50%

解决方案: 不再复制模型, 而是把模型本身拆开 -> 张量并行 + 流水线并行

课时2

模型并行: 把模型拆开

Tensor Parallelism / Pipeline Parallelism / Sequence Parallelism

上节回顾: ZeRO解决了内存冗余

为什么: ZeRO把16x冗余降到了N分之一, 但还有两个挑战没解决

两个新问题

问题 1: 单层激活可能超过显存

- Attention的内存 = $O(s^2 * h)$
- GPT-3: $s=2048, h=12288$ -> 单层attention ~48GB
- 长序列(128K): 远超80GB显存

问题 2: ZeRO-3通信量增加50%

- AllGather(W_fwd) + ReduceScatter(G) + AllGather(W_bwd)
- 每层3次集合通信 vs DP的2次
- 跨节点带宽成为瓶颈

解决方案

不复制模型, 而是拆分模型!

Tensor Parallelism (TP)

-> 层内拆分: 把权重矩阵切开

Pipeline Parallelism (PP)

-> 层间拆分: 不同GPU放不同层

Sequence Parallelism (SP)

-> 序列拆分: 沿token维度切分

Key: 数据并行复制模型 -> 模型并行拆分模型。TP切层内权重, PP切层间, SP切序列维度。

本节课路线图: 三种模型并行

为什么: 不同并行策略适用于不同瓶颈场景, 理解全貌才能组合使用

PART 05

Tensor Parallelism

张量并行

- 层内权重拆分
- Column/Row Parallel
- MLP + Attention TP
- 通信量分析
- 限制: 节点内

PART 06

Pipeline Parallelism

流水线并行

- 层间拆分
- GPipe micro-batch
- 1F1B schedule
- Zero Bubble PP
- 跨节点部署

PART 07

Sequence Parallelism

序列并行

- 序列维度拆分
- Megatron SP
- Ring Attention
- Ulysses
- 百万级context

Key: TP解决单层大问题(节点内), PP解决层数多问题(跨节点), SP解决长序列问题。

PART 05

张量并行 (Tensor Parallelism)

把每一层的权重矩阵切开, 多张GPU协同计算一次矩阵乘法

TP核心思想: 切分权重矩阵

为什么: 单层参数矩阵太大(GPT-3 MLP: 12288x49152), 一张卡放不下或算不动

核心公式: $Y = XW$

把W按列/行切分到t张GPU:

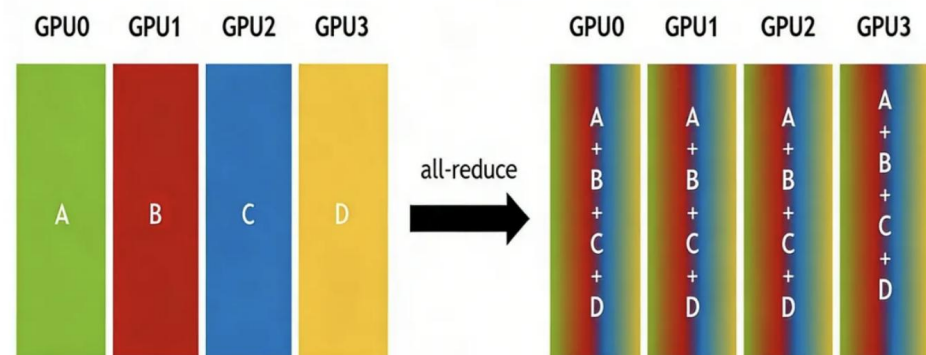
Column split: $W = [W1 \mid W2 \mid \dots \mid Wt]$

-> GPU_i computes $Y_i = X * W_i$

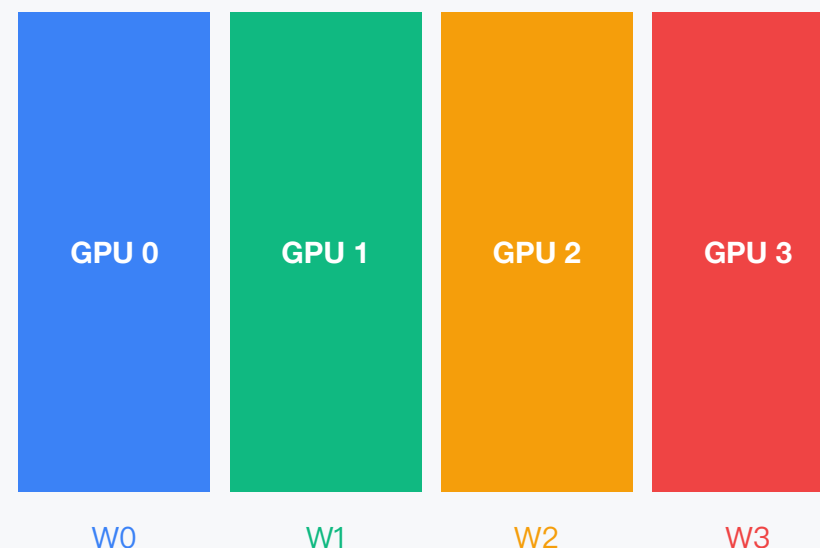
Row split: $W = [W1; W2; \dots; Wt]$

-> GPU_i computes $Y_i = X_i * W_i$

-> AllReduce: $Y = \text{sum}(Y_i)$



Weight Matrix W (12288 x 49152)



每张卡: 12288 x 12288

内存: $49152^2 \times 2B = 4.6GB \rightarrow$ 每卡1.15GB

Key: TP的本质: 矩阵乘法天然可分 -- 按列切输出独立, 按行切需要AllReduce求和。

MLP Column-Parallel: GeLU不需要通信!

为什么: MLP第一层用列切分, 因为GeLU是element-wise操作, 各GPU可独立计算

MLP公式:

$$Y = \text{GeLU}(XA) * B$$

Step 1: 对A按列切分

$$A = [A1 \mid A2] \quad (t=2 \text{ GPUs})$$

Step 2: 每张GPU独立计算

$$\text{GPU}_i: Z_i = X * A_i$$

Step 3: GeLU是element-wise!

$$\text{GeLU}([Z1 \mid Z2]) = [\text{GeLU}(Z1) \mid \text{GeLU}(Z2)]$$

-> 不需要跨GPU通信!

X (input, replicated)

GPU0: $X * A1 \rightarrow Z1$

GPU1: $X * A2 \rightarrow Z2$



-> Ready for Row-Parallel B

Key: 列切分 + element-wise激活函数 = 零通信! 这是TP效率的关键设计。

MLP Row-Parallel: AllReduce求和

为什么: B矩阵按行切分, 每GPU计算部分结果, AllReduce汇总得到最终输出

```
class ColumnParallelLinear(nn.Module):
    def __init__(self, in_f, out_f, world, rank):
        super().__init__()
        # 只存 out_f/world 列
        self.weight = nn.Parameter(
            torch.empty(out_f // world, in_f))
        nn.init.kaiming_uniform_(self.weight)

    def forward(self, x):
        return F.linear(x, self.weight)
        # 无需通信!

class RowParallelLinear(nn.Module):
    def __init__(self, in_f, out_f, world, rank):
        super().__init__()
        # 只存 in_f/world 行
        self.weight = nn.Parameter(
            torch.empty(out_f, in_f // world))

    def forward(self, x):
        out = F.linear(x, self.weight)
        dist.all_reduce(out) # AllReduce!
        return out
```

Row-Parallel原理

B按行切分:

$B = [B_1; B_2; \dots; B_t]$

GPU_i计算:

$Y_i = \text{GeLU}(Z_i) * B_i$

最终结果:

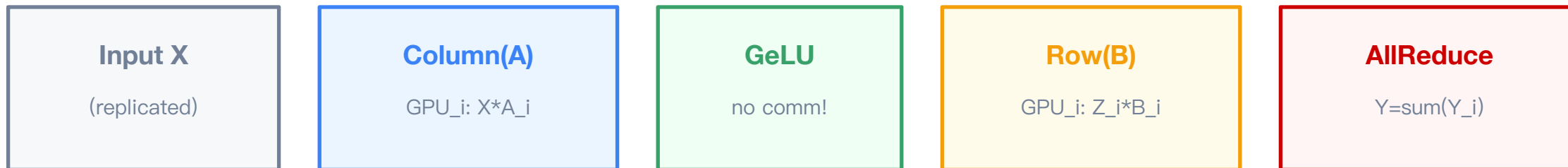
$Y = \text{sum}(Y_i)$

-> AllReduce!

Key: Column-Parallel(A) + GeLU + Row-Parallel(B) = 1次AllReduce完成整个MLP。

完整MLP张量并行流程

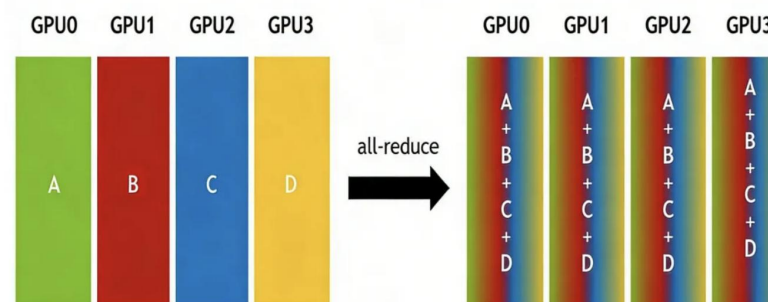
为什么: 理解端到端数据流和通信点, 才能分析TP的通信开销



Forward: 1x AllReduce (after Row-Parallel)

Backward: 1x AllReduce (gradient of Column-Parallel input)

Total per MLP layer: **2x AllReduce (1 fwd + 1 bwd)**



对比: 如果不用TP, 每GPU要存完整的A(12288x49152)和B(49152x12288)。TP=8时每卡只存1/8。

Key: MLP TP = Column(A) -> GeLU(free) -> Row(B) -> AllReduce。每MLP层2次AllReduce。

Attention TP: Multi-Head天然可并行

为什么: **Multi-head attention**本身就是多头独立计算, 天然适合按**head**拆分

```
# TP Attention (t=4, h=32 -> 8 heads/GPU)
class TPAttention(nn.Module):
    def __init__(self, d_model, n_heads, tp_size):
        super().__init__()
        self.local_heads = n_heads // tp_size
        self.head_dim = d_model // n_heads
        # 只存 local_heads 个head的投影
        self.wq = ColumnParallelLinear(
            d_model, self.local_heads*self.head_dim)
        self.wk = ColumnParallelLinear(...)
        self.wv = ColumnParallelLinear(...)
        # Output: Row-Parallel
        self.wo = RowParallelLinear(
            self.local_heads*self.head_dim, d_model)

    def forward(self, x):
        q = self.wq(x) # 只算local heads
        k = self.wk(x)
        v = self.wv(x)
        out = attention(q, k, v)
        out = self.wo(out) # Row-parallel
        dist.all_reduce(out) # sync
        return out
```

为什么Attention适合TP?

1. Multi-head本质是独立计算
h个head互不依赖
2. 按head分配到GPU
32 heads / 4 GPUs = 8/GPU
3. Q/K/V投影: Column-Parallel
(无需通信)
4. Output投影: Row-Parallel
+ AllReduce

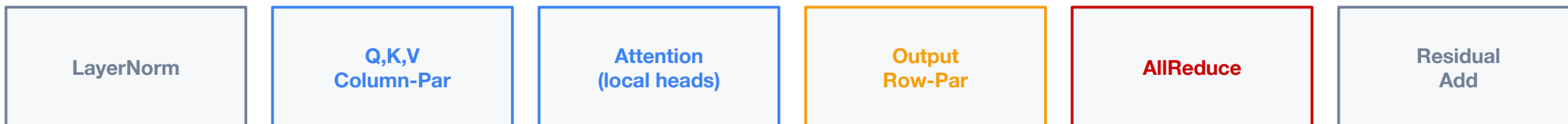
通信: **1x AllReduce (fwd)**

Key: Attention的multi-head结构天然适合TP: h/t heads per GPU, 只需1次AllReduce。

完整Transformer层的TP数据流

为什么: 一个Transformer层 = Attention + MLP, 共需4次AllReduce (fwd+bwd)

Attention Block



MLP Block



每个Transformer层 (Forward): **2x AllReduce** (Attention 1次 + MLP 1次)

每个Transformer层 (Backward): **2x AllReduce** (梯度传播对称)

Key: 1个Transformer层 = 4次AllReduce (2 fwd + 2 bwd)。通信量直接决定TP效率。

TP通信量分析: 真实数字

为什么: 量化通信开销才能判断TP的适用范围和GPU数量上限

每次AllReduce通信量 = $2 * (t-1)/t * b * s * h$ bytes (ring AllReduce)

Parameter	GPT-3 175B	LLaMA-70B
Hidden dim (h)	12,288	8,192
TP degree (t)	8	8
Batch (b) x Seq (s)	4 x 2048	4 x 4096
Per AllReduce (fp16)	384 MB	512 MB

GPT-3计算过程:

每次AR = $2 * (8-1)/8 * 4 * 2048 * 12288$
= $1.75 * 100,663,296$
= 176M elements
= 352MB (fp16)

每层4次AR = ~1.4GB通信/layer

GPT-3: 96层 x 4次AR = **384次AllReduce/step**, 总通信 ~135GB/step

Key: TP通信量 = $O(b*s*h)$ per AllReduce, 每层4次。GPT-3: 384次AR/step, ~135GB通信。

为什么TP只能在节点内?

为什么: 384次AllReduce/step的带宽需求, 只有NVLink能满足

带宽需求 vs 可用带宽

互连	带宽	384次AR耗时
NVLink (H100)	900 GB/s	~0.15s
PCIe Gen5	128 GB/s	~1.1s
InfiniBand HDR	50 GB/s	~2.7s
RoCE 100G	12.5 GB/s	~10.8s

计算时间: GPT-3 per step ~30-60ms (compute)

-> IB: 通信/计算 = 2.7s/0.06s = 45x overhead!

TP的硬件约束

1. 通信频率极高

每层4次AllReduce, 不可overlap
(在计算图的关键路径上)

2. 延迟敏感

AllReduce有同步barrier
跨节点延迟 >> 节点内延迟

3. 实践原则:

TP degree <= 同一节点GPU数
通常 t = 4 或 8

Key: TP受NVLink带宽限制, 只能在节点内(t<=8)。跨节点需要Pipeline Parallelism。

Megatron-LM TP实践

为什么: Megatron-LM是TP的标准实现, 理解其配置才能正确使用

```
# Megatron-LM TP配置
args.tensor_model_parallel_size = 8 # TP度
# 自动创建通信组:
# TP group: [0,1,2,3,4,5,6,7] (node内)
# DP group: [0,8,16,...] (跨节点)

# 2D并行: TP x DP
# 16 GPUs (2 nodes x 8 GPUs):
#   TP groups: [0-7], [8-15]
#   DP groups: [0,8], [1,9], ...

# 启动命令
torchrun --nproc-per-node=8 \
  --nnodes=2 \
  pretrain_gpt.py \
  --tensor-model-parallel-size 8 \
  --pipeline-model-parallel-size 1 \
  --micro-batch-size 1 \
  --global-batch-size 64
```

Megatron-LM要点

1. TP组 = 节点内GPU

利用NVLink高带宽

2. DP组 = 跨节点相同rank

GPU[0]和GPU[8]做DP

3. 自动权重切分

nn.Linear自动替换为
Column/RowParallelLinear

4. 兼容PP和SP

可与PP/SP组合使用

Shoeybi, Mohammad, et al. "Megatron-lm: Training multi-billion parameter language models using model parallelism." arXiv preprint arXiv:1909.08053 (2019).

Key: Megatron-LM: TP组(节点内NVLink) + DP组(跨节点IB), 自动替换nn.Linear。

TP的局限 -> 需要Pipeline Parallelism

为什么: TP受限于单节点GPU数(8), 超大模型需要跨节点拆分 → 引出PP

TP的天花板

TP degree ≤ 8 (单节点GPU数)

GPT-3 175B: 每卡最少需要多少内存?

- 参数: $175B * 2B = 350GB$
- TP=8: $350/8 = 43.75GB$ (参数)
- + 优化器状态 + 梯度 + 激活
- 总计 >> **80GB (A100)**

TP=8还不够! 8张GPU放不下175B!

而且TP=8已经是NVLink的极限

答案: 层间拆分 (PP)

Pipeline Parallelism:

把不同的层放在不同的节点!

GPT-3: 96层

PP=8: 每个stage 12层

-> 参数/8, 激活/8

通信模式:

- 只需P2P传递激活(不是AllReduce)
- 只在相邻stage之间
- 通信量远小于TP

但: 有流水线气泡(bubble)问题!

Key: TP ≤ 8 不够, PP跨节点切层解决。代价: 流水线气泡。下面详细分析PP。

PART 06

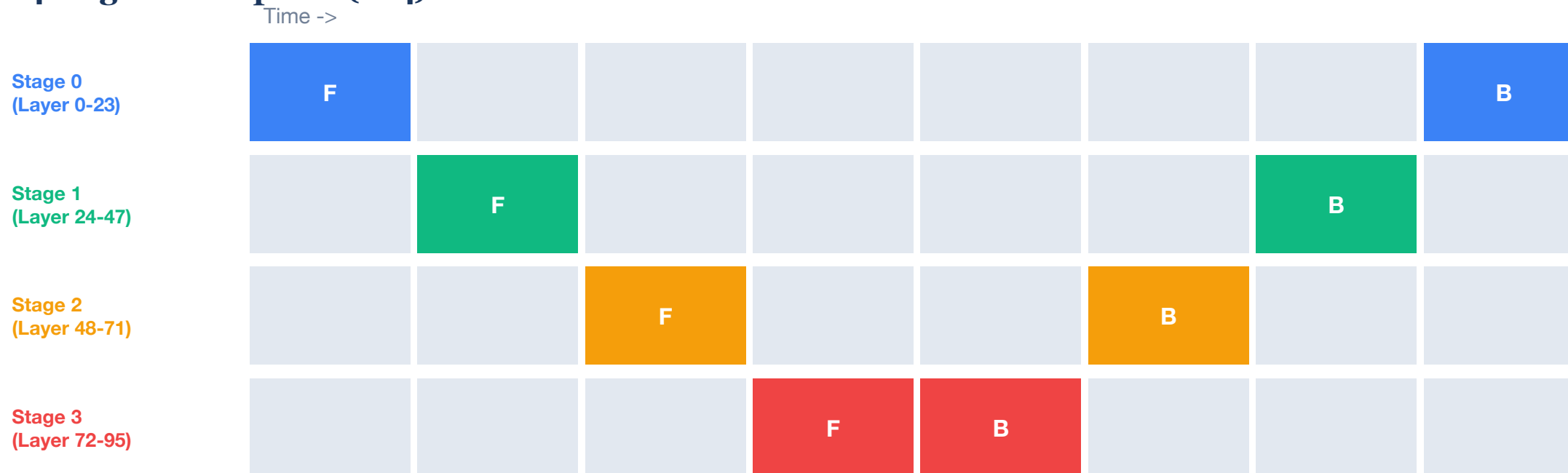
流水线并行 (Pipeline Parallelism)

把模型的不同层放在不同的GPU/节点上, 流水线执行

Naive Pipeline: 巨大的气泡浪费

为什么: 最简单的PP -- 串行执行, 导致P-1张GPU空闲, 利用率仅1/P

4-Stage Naive Pipeline (P=4)



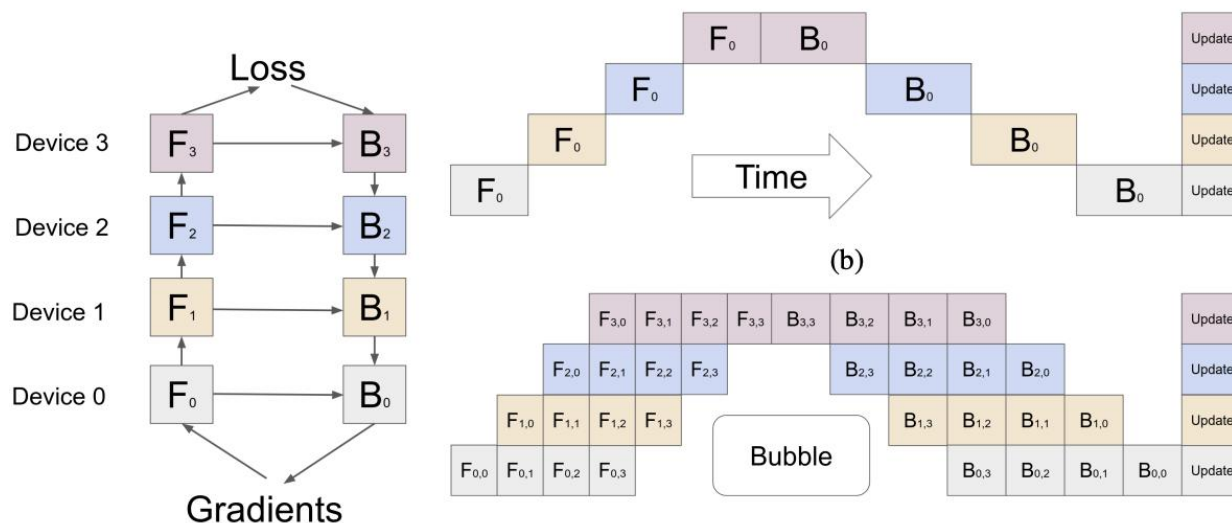
Bubble ratio = $(P-1)/P = 75\%$ ($P=4$)。只有1张GPU在任意时刻干活! 效率极低。

Key: Naive PP: bubble ratio = $(P-1)/P$ 。P=4时75%浪费。解决方案: micro-batching。

GPipe: Micro-Batch流水线

为什么: 把mini-batch拆成m个micro-batch, 让流水线填满, 减少bubble

GPipe: $P=4$ stages, $m=8$ micro-batches



Huang, Yanping, et al. "Gpipe: Efficient training of giant neural networks using pipeline parallelism." Advances in neural information processing systems 32 (2019).

Bubble ratio = $(P-1) / (m + P-1)$

$P=4, m=8$: bubble = $3/11 = 27.3\%$ (vs naive 75%!)

m越大 -> bubble越小, 但activation内存 $O(m)$ 增长

Key: GPipe用m个micro-batch填满流水线。Bubble = $(P-1)/(m+P-1)$, $m=8$ 时27.3%。

Bubble比例 vs Micro-Batch数量

为什么: 量化m的选择对效率和内存的影响, 找到最优平衡点

m (micro-batches)	Bubble Ratio	GPU利用率	Activation内存
m = 1 (naive)	75.0%	25.0%	1x
m = 4	42.9%	57.1%	4x
m = 8	27.3%	72.7%	8x
m = 16	15.8%	84.2%	16x

m = 32: bubble = 8.6%, 利用率 = 91.4%, 但activation内存 = 32x!

Trade-off: m大 -> bubble小, 但需要缓存m份激活。解决: activation checkpointing (牺牲33%计算换内存)。

实践: Megatron-LM默认 $m = \text{global_batch} / (\text{DP} * \text{PP})$, 配合activation checkpointing使用。

Key: m=16-32是常用范围: bubble<16%, 配合activation checkpointing控制内存。

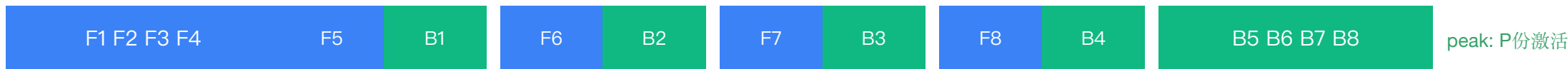
1F1B Schedule: 降低峰值内存

为什么: GPipe需缓存m份激活($O(m)$), 1F1B warm-up后交替执行, 峰值内存降到 $O(P)$

GPipe: All Forward -> All Backward (peak memory = $O(m)$)



1F1B: Warm-up -> Alternate 1F+1B -> Cool-down (peak memory = $O(P)$)



	GPipe	1F1B (PipeDream)
Peak Memory	$O(m)$ micro-batch activations	$O(P)$ stage activations
Bubble Ratio	$(P-1)/(m+P-1)$	$(P-1)/(m+P-1)$ (same!)
Gradient Sync	Sync (consistent)	Sync (flush at end)

Guan, Lei, et al. "PipeOptim: Ensuring effective 1F1B schedule with optimizer-dependent weight prediction." IEEE Transactions on Knowledge and Data Engineering (2025).

Key: 1F1B: warm-up P步后交替1F+1B, 峰值内存从 $O(m)$ 降到 $O(P)$, bubble不变。

Interleaved Schedule: 进一步减少Bubble

为什么: 每GPU持有v个不连续chunk, bubble降v倍, 代价是v倍P2P通信

核心思想: 每张GPU持有v个不连续的layer chunks

例: 16层, 4 GPUs, v=2

GPU0: layers 0-1, 8-9 (chunk 0, chunk 4)

GPU1: layers 2-3, 10-11 (chunk 1, chunk 5)

GPU2: layers 4-5, 12-13 (chunk 2, chunk 6)

GPU3: layers 6-7, 14-15 (chunk 3, chunk 7)

Bubble ratio:

$$(P-1) / (m*v + P-1)$$

$$v=1: (P-1)/(m+P-1)$$

v=2: bubble减半

v=4: bubble减到1/4!

代价: v倍P2P通信

v	Bubble (P=4,m=8)	P2P comm	Megatron-LM配置
1	27.3%	1x	num_layers_per_vp_stage = L/P
2	15.0%	2x	num_layers_per_vp_stage = L/(P*2)
4	8.6%	4x	num_layers_per_vp_stage = L/(P*4)

Key: Interleaved: v个virtual stages, bubble减v倍, P2P通信增v倍。v=2-4常用。

Zero Bubble Pipeline Parallelism

为什么: 拆分Backward为B(input grad)+W(weight grad), W可填充bubble

关键洞察: **Backward**可以拆成两部分!

B (input gradient): $dL/dX = dL/dY * W^T$

-> 依赖上游梯度, 在关键路径上

W (weight gradient): $dL/dW = X^T * dL/dY$

-> 只需本地数据, 可以延迟执行! 用来填**bubble**!

ZB-H1 (Heuristic)

- 1F1B基础上, 把W移到bubble位置
- 接近零气泡 (**bubble < 1%**)
- 无额外内存开销
- 梯度同步一致(synchronous)
- Megatron-LM已集成

ZB-V (Optimal)

- ILP求解最优调度
- 理论零气泡!
- 需要slightly更多内存
- 自动生成调度方案
- *Paper: ICLR 2024 Spotlight*

Key: Zero Bubble PP: 把Backward拆为B+W, W不在关键路径 -> 填充bubble -> 接近零浪费。

PP通信: P2P vs TP的AllReduce

为什么: PP只需相邻stage间P2P传激活, 通信量远小于TP, 适合跨节点

	Tensor Parallelism	Pipeline Parallelism
通信模式	AllReduce (all-to-all)	P2P Send/Recv
通信次数/step	4 x L (每层4次AR)	2 x m (fwd+bwd各m次)
每次通信量	$2(t-1)/t * b*s*h$ bytes	$b*s*h$ bytes (激活)
GPT-3总通信	~135GB/step	~3.2GB/step

PP通信量计算 (GPT-3):

每次P2P = $b*s*h * 2B$

= $4*2048*12288*2 = 200MB$

m=16: $16*2 = 32$ 次 -> 6.4GB/step

PP通信特点:

1. 通信量~TP的1/40
 2. 只涉及2个GPU(P2P)
 3. 可与计算overlap
- > 适合跨节点(IB 50GB/s足够)

Key: PP通信 = P2P激活传递, 量级远小于TP的AllReduce。PP适合跨节点, TP限节点内。

PP配置: Megatron-LM & DeepSpeed

为什么: 掌握实际框架配置, 才能在真实训练中使用PP

```
# Megatron-LM PP配置
args.pipeline_model_parallel_size = 8 # PP度
# 1F1B + interleaved:
args.num_layers_per_virtual_pipeline_stage = 2
# micro-batch设置:
args.micro_batch_size = 1
args.global_batch_size = 1024
# 自动计算 m = 1024 / (DP * PP * micro_bs)

# 启动: 64 GPUs (8 nodes x 8 GPUs)
# TP=8 (node内), PP=8 (跨节点)
torchrun --nproc-per-node=8 \
  --nnodes=8 \
  pretrain_gpt.py \
  --tensor-model-parallel-size 8 \
  --pipeline-model-parallel-size 8
```

```
# DeepSpeed PP (JSON config)
{
  "train_batch_size": 1024,
  "train_micro_batch_size_per_gpu": 1,
  "steps_per_print": 10,
  "pipeline": {
    "enabled": true,
    "stages": 8,
    "schedule": "1f1b"
  }
}
```

配置要点:

- PP组跨节点, TP组在节点内
- micro_batch_size尽量小(=1)
- interleaved需 L可被P*v整除

Key: 实践: TP=8(节点内NVLink) + PP=8(跨节点IB) = 64 GPUs训练175B模型。

PP决策框架: 什么时候用PP?

为什么: PP引入bubble开销, 需要权衡利弊决定是否使用以及如何配置

维度	PP的代价	PP的收益
计算效率	Bubble浪费 8-27%	每卡计算量减P倍
内存	m份micro-batch激活缓存	参数/梯度/优化器减P倍
通信	P2P延迟(相邻stage)	比TP低40x (P2P vs AR)
实现复杂度	调度逻辑复杂	不改模型代码(切层即可)

决策指南

- 模型放不下单节点(TP=8不够) -> **必须用PP**
- 跨节点带宽有限($IB < 100GB/s$) -> **PP优于TP**
- GPU数 ≤ 8 且NVLink可用 -> **TP优于PP (无bubble)**

Key: PP适合跨节点大模型训练。典型: TP=8(节点内) + PP=N(跨节点) + DP=剩余GPU。

PART 07

序列并行 -- 解决长序列问题

Attention内存 $O(s^2)$, 长序列(128K+)一层就超80GB显存

为什么需要序列并行?

为什么: Attention的激活内存 $O(s^2 \cdot h)$, 长序列场景下单层就超过GPU显存

Attention激活内存计算

Attention Score矩阵: $Q * K^T$

Shape: $[b, h_heads, s, s]$

内存 = $b * n_heads * s^2 * 2B$

例: LLaMA-70B, $s=128K$, $h=8192$

$n_heads=64$, $b=1$

= $1 * 64 * 128K^2 * 2B$

= $64 * 16.4B * 2B$

= 2TB (单层!)

>> 80GB (A100/H100)

Attention内存随序列长度增长

Seq Length	Attn Memory	Status
2K	0.5 GB	OK
8K	8 GB	OK
32K	128 GB	> 80GB!
128K	2 TB	25x OOM!

$O(s^2)$ = 灾难!

即使用了TP=8:

128K: $2TB/8 = 250GB$ >> 80GB

-> 必须沿序列维度拆分!

Key: Attention内存 $O(s^2)$: 128K context单层需2TB。TP不够, 必须序列并行。

Megatron SP: 补全TP的冗余计算

为什么: TP中LayerNorm/Dropout在每GPU上重复计算, SP沿序列维切分消除冗余

TP中的冗余问题:

TP对MLP和Attention做了切分
但 LayerNorm / Dropout / Residual Add
每张GPU都计算完整的 **[b, s, h]**!
-> 冗余计算 + 冗余内存

Megatron SP解决方案:

对non-TP ops沿sequence维切分:
GPU_i处理 token $[i*s/t : (i+1)*s/t]$
LayerNorm: $[b, s/t, h]$ instead of $[b, s, h]$

通信模式变化

TP Only (before):

AllReduce = AllGather + ReduceScatter
(合成一个操作, 通信 $2*(t-1)/t$)

TP + SP (after):

把AllReduce拆开:

- ReduceScatter (进入SP区域)
 - > 各GPU得到不同token段的完整结果
- AllGather (进入TP区域)
 - > 恢复完整序列用于矩阵乘法

总通信量不变! 但激活内存减为 $1/t$

Key: Megatron SP: AllReduce拆为ReduceScatter+AllGather, non-TP ops沿seq切分, 激活内存 $/t$ 。

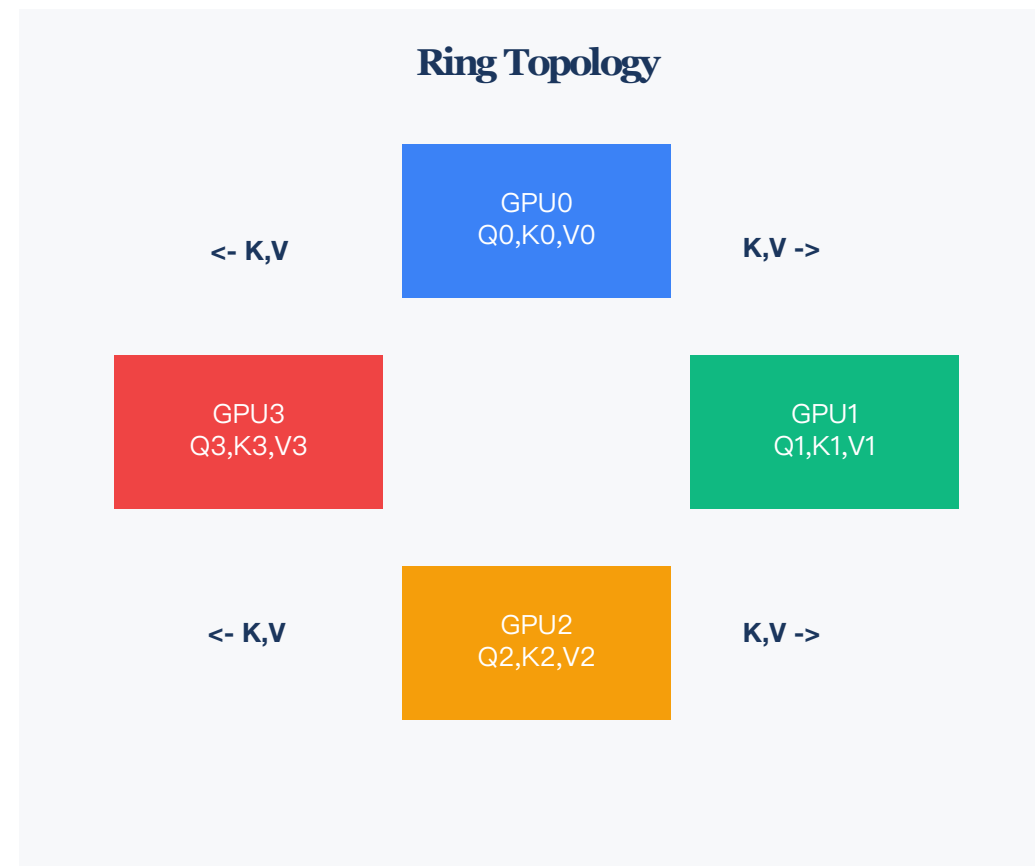
Ring Attention: 支持百万级Token

为什么: Megatron SP只切non-TP ops; Ring Attention直接切Attention的QKV计算

核心思想:

1. 每GPU持有 s/P 个token的Q, K, V
2. K, V沿GPU环形传递
每步收到新的K,V块, 计算部分attention
3. P步后, 每个Q看过所有K,V
使用online softmax (FlashAttention技巧)
4. 通信与计算overlap!
当前K,V计算时, 下一组K,V在传输

内存: $O(s^2/P)$ per GPU
P=8: 128K $\rightarrow$ 每卡等效16K序列



Key: Ring Attention: KV环形传递+online softmax, 通信overlap计算, 支持百万token。

Ulysses vs Ring Attention

为什么: 两种SP方案切法不同, 适用场景不同, 需要对比选择

DeepSpeed Ulysses

1. 输入沿seq维切分: $[b, s/P, h]$
2. AlltoAll: Q,K,V $\rightarrow$ head-parallel
每GPU: $[b, s, h/P]$ (所有token, 部分head)
3. 标准Attention (每GPU在自己的heads上)
4. AlltoAll: 结果 $\rightarrow$ seq-parallel back

Ring Attention

1. 输入沿seq维切分: $[b, s/P, h]$
2. K,V环形传递P步
每步计算部分attention score
3. Online softmax累积最终结果
4. 通信可完全与计算overlap

	Ulysses	Ring Attention
切分维度	Seq $\rightarrow$ Head parallel	Seq parallel throughout
通信原语	2x AlltoAll	P步 P2P (ring)
Overlap可能	有限 (AlltoAll是blocking)	完全overlap
SP扩展性	$P \leq n_heads$	P可以很大

Key: Ulysses: AlltoAll做head-parallel, 适合中等序列; Ring: P2P环形, overlap好, 适合超长序列。

本节课总结: TP / PP / SP 全景对比

为什么: 理解三种模型并行的定位和互补关系, 为下节课的混合并行做准备

	Tensor Parallel	Pipeline Parallel	Sequence Parallel
切分维度	层内权重 (column/row)	层间 (stage)	序列 (token)
通信模式	AllReduce (4/layer)	P2P (2m/step)	AlltoAll/Ring P2P
适用范围	节点内 (NVLink)	跨节点 (IB)	长序列 (>32K)
代价	高频通信	Pipeline bubble	AlltoAll开销

我们有了 DP, TP, PP, SP 四种武器。

下节课:

- 1. 怎么组合? 3D/4D/5D Parallelism
- 2. 怎么选最优配置? Auto-parallelism
- 3. 推理也需要并行吗? Inference Parallelism

Key: 模型并行三件套: TP(层内/节点内) + PP(层间/跨节点) + SP(序列/长context)。下节课: 组合策略。

课时3

混合并行与并行推理: 组合与工程实践

Hybrid Parallelism & Parallel Inference: Engineering Practice at Scale

回顾与本节核心问题

从单一策略到组合拳: 前两节给了我们四把武器, 本节学习如何排兵布阵

前两节回顾

课时1: 通信原语, DDP, ZeRO, FSDP

> 复制模型拆数据, 解决内存冗余

课时2: TP(拆层内权重), PP(拆层间), SP(拆序列)

> 解决单层/单模型放不下的问题

共同局限: 单一策略无法高效扩展到1000+ GPU

本节三大问题

Q1: 怎么组合 DP+TP+PP+SP+EP?

> 3D/5D 混合并行

Q2: 能否自动搜索最优并行策略?

> Alpa, Galvatron, FlexFlow

Q3: 推理也需要并行吗?

> 投机解码, 解聚推理, 分布式vLLM

Key Takeaway: 本节从"单一维度"进入"多维组合", 完成从训练并行到推理并行的全景图

PART 08

混合并行：实战中的组合

3D/5D Parallelism: Combining Strategies for 1000+ GPU Scale

为什么需要混合并行?

每种策略都有天花板, 单一策略无法扩展到千卡以上规模

策略	核心限制	可扩展GPU数	瓶颈
TP (Tensor)	每层2次AllReduce, 需NVLink	<=8	通信带宽
PP (Pipeline)	Pipeline bubble浪费	4-32	bubble + 不均衡
DP (Data)	每卡需完整模型副本	无上限	内存冗余
SP (Sequence)	仅长序列场景有效	取决于seq_len	额外通信

核心矛盾: 训练GPT-3 175B需要1024张A100, 但:

- TP最多8卡(NVLink拓扑限制), 只解决单层放不下
- PP有bubble开销, 且流水线级数不宜过多(>16级bubble严重)
- 纯DP要求每卡放完整模型, 175B根本放不下
- **解法:** 分层组合, 通信频率匹配网络带宽层次

Key Takeaway: 单一并行策略有扩展天花板, 混合并行通过"通信-带宽匹配"突破千卡规模

3D并行: DP + TP + PP

核心原则: 通信频率高的放在带宽大的链路上

Tier 1: NVLink (600 GB/s)

TP = 8

每层2次AllReduce
通信最频繁 -> 最快链路

V

Tier 2: InfiniBand (200 Gb/s)

PP = 8-16

微批次间P2P传激活值
频率中等 -> 节点间高速网

V

Tier 3: Ethernet / IB (跨机架)

DP = $N / (TP * PP)$

每步1次AllReduce梯度
频率最低 -> 跨机架

Total GPUs = TP x PP x DP (e.g., 8 x 8 x 16 = 1024)

Key Takeaway: 3D并行 = TP(NVLink内) + PP(节点间IB) + DP(跨机架), 匹配三级网络拓扑

案例: GPT-3 175B 训练配置

Megatron-LM: 1024 x A100, TP=8, PP=8, DP=16 的设计推导

配置推导过程

1. 模型大小: 175B params = ~350 GB (FP16)
2. 训练状态: 参数+梯度+优化器 = ~3.5 TB
3. A100 80GB, 单卡放不下 -> 需要模型并行
4. TP=8: 8卡NVLink组, $350\text{GB}/8 = \sim 44\text{ GB/卡}$

加上激活值, 刚好80GB内可行

5. PP=8: $96\text{层}/8 = 12\text{层/stage}$

每stage参数 $\sim 350/8 = \sim 44\text{ GB}$

6. DP=1024/(8x8)=16: 数据并行扩吞吐

7. 每卡实际内存: $\sim 44\text{GB参数} + \text{激活值}$

+ ZeRO-1优化器状态/16 $\sim 70\text{GB}$

GPT-3 175B Config

Params: 175B (96 layers, h=12288)
GPU: 1024 x A100 80GB
TP=8 (NVLink, 每层2x AllReduce)
PP=8 (1B, 1F1B schedule)
DP=16 (跨机架, gradient AR)

训练效率指标

- MFU (Model FLOPs Utilization): ~47%
- 训练时长: ~34天 (1024 A100)
- 通信开销: TP ~15%, PP ~8%, DP ~5%
- Pipeline bubble: ~12% (1F1B, micro=32)
- 总成本: ~\$4.6M (按2023云价格)

Key Takeaway: GPT-3配置的核心逻辑: 先用TP填满NVLink组, 再用PP拆层, 最后DP扩吞吐

带宽匹配分析: 通信量 vs 链路带宽

定量分析: 每个并行维度的通信频率和数据量, 匹配最佳网络层次

维度	通信操作	频率/步	数据量(175B)	需求带宽
TP	2x AllReduce / layer	96x2 = 192次	$2*b*s*h = \sim 1.6\text{GB/次}$	NVLink 600GB/s
PP	P2P send/recv	m次 (micro-batch)	$b*s*h = \sim 0.8\text{GB/次}$	IB 200Gb/s
DP	AllReduce gradients	1次/step	$\sim 350\text{GB} / \text{DP_size}$	跨机架 IB/ETH

匹配逻辑:

- TP: 每层前向+反向各2次AllReduce, 总192次/step -> 必须用NVLink (600GB/s)
- PP: 微批次间传激活值, m次P2P (m=32时32次) -> IB够用 (25GB/s)
- DP: 整步只1次梯度AllReduce, 可overlap with backward -> 跨机架

关键洞察:

如果把TP放到IB上, 通信开销增加6-8x, 训练效率骤降至<30%

Key Takeaway: TP>PP>DP的通信频率递减, 必须严格匹配NVLink>IB>Ethernet的带宽层次

案例: LLaMA-3 405B 训练配置

Meta 2024: 更大模型 + 更长上下文 = 新的并行维度 (Context Parallelism)

配置项	GPT-3 175B	LLaMA-3 405B	差异分析
参数量	175B	405B	2.3x更大
上下文长度	2K	128K	64x更长!
TP	8	8	NVLink限制不变
PP	8	16	模型更大, 更多stage
CP (Context)	-	8	新维度! 拆序列

为什么需要 Context Parallelism (CP)?

- 128K context时, attention的激活值 = $O(s^2) = 128K^2 = 16G$ tokens -> 单卡放不下
- CP=8: 每卡处理 $128K/8=16K$ tokens, 用Ring Attention交换KV
- 总GPU: $TP(8) \times PP(16) \times CP(8) \times DP(N) = 1024+$ GPUs on H100 cluster
- 训练: **16K x H100, ~54天, 3.8×10^{25} FLOPs**

Key Takeaway: LLaMA-3引入CP维度解决长上下文激活值爆炸, 混合并行从3D升级到4D

案例: BLOOM-176B (Megatron-DeepSpeed)

BigScience 2022: 768 x A100, 开源社区的工程实践经验

BLOOM-176B Configuration

Framework: Megatron-DeepSpeed

GPU: 768 x A100 80GB (48 nodes)

TP = 4 (NVLink 4-way per node)

PP = 12 (70 layers / groups)

DP = 16 ($768 / (4 \times 12) = 16$)

ZeRO: Stage 1 (optimizer shard)

Precision: BF16 + loss scaling

Duration: ~3.5 months

为什么 TP=4 而非 TP=8?

A100 NVLink拓扑差异:

- DGX A100: 8卡全连接NVLink -> TP=8
- Jean Zay集群: 4卡/组 NVLink -> TP=4
- TP组内通信远大于组间 -> 必须匹配!

工程教训:

- 硬件拓扑决定TP度数, 不是"越大越好"
- PP=12偏多(bubble~16%), 用interleaved schedule缓解

BLOOM vs GPT-3 对比启示: 同规模模型, 不同硬件拓扑 -> 不同的最优配置。没有"万能配方", 必须依据硬件定制。

Key Takeaway: BLOOM证明: TP度数取决于NVLink拓扑而非经验值, 硬件差异直接改变最优并行策略

Expert Parallelism: 通信模式分析

Ch4已讲MoE路由/负载均衡, 这里聚焦EP的通信开销与扩展性

EP通信模式: 每MoE层2次AlltoAll

1. Router选择expert (本地计算)

2. AlltoAll Dispatch: 发送token到对应expert所在GPU

3. Expert计算 (每GPU上独立FFN)

4. AlltoAll Collect: 收回计算结果到原始GPU

通信量分析:

- 每次AlltoAll: $b*s*h$ bytes (top-2路由)
- Mixtral: $b=1, s=4K, h=4096 \rightarrow \sim 32MB/次$
- 32层MoE $\rightarrow 64$ 次AlltoAll/step
- 远超TP的AllReduce!

EP的扩展性挑战

AlltoAll vs AllReduce:

AllReduce: $O(M)$ 数据量, 与GPU数无关
AlltoAll: $O(M*N/p)$ $\rightarrow$ GPU越多通信越碎

缓解策略:

- Hierarchical AlltoAll: 先组内再组间
- Expert Slicing: 每个expert拆到多GPU
- Token Dropping: 牺牲精度减通信
- DeepSeek-V2: 用共享expert减少跨GPU通信的token比例

Key Takeaway: EP的AlltoAll通信是MoE训练的核心瓶颈, 通信量随expert分散程度增加

5D并行: DP + TP + PP + SP + EP

MoE大模型的完整并行方案: 每个维度解决不同瓶颈

DP

扩吞吐, 拆数据

TP

拆层内权重, NVLink通信

PP

拆层间, 减单卡层数

SP/CP

拆序列, 解决长上下文

EP

拆expert, MoE专用

实际配置示例

```
# DeepSpeed MoE Config (示意)
ds_config = {
    "train_micro_batch_size_per_gpu": 1,
    "tensor_parallel": {"tp_size": 8},
    "pipeline_parallel": {"pp_size": 4},
    "expert_parallel": {"ep_size": 8, "num_experts": 64},
    "sequence_parallel": True,      # SP跟随TP
    "data_parallel": "auto",        # = total/(TP*PP*EP)
    "zero_optimization": {"stage": 1}, # ZeRO-1 for optimizer
}
# Mixtral 8x7B: TP=8, PP=1, EP=8 -> 每GPU 1 expert
```

Key Takeaway: 5D并行 = 3D + SP + EP, 每个维度独立正交, 总GPU = TP x PP x DP x EP (SP随TP)

千卡训练的工程挑战

1000+ GPU规模: 平均每小时1次故障, 工程可靠性成为核心问题

核心挑战

1. 故障容错 (Fault Tolerance)

- GPU故障率: ~0.1%/天/卡 -> 1000卡 ~1次/天
- Checkpoint: 每N步保存, 异步写入
- Meta LLaMA-3: 每小时checkpoint, 恢复<10min

2. 弹性训练 (Elastic Training)

- 故障节点踢除, 不中断训练
- 动态调整DP degree

3. 集群调度

- 网络拓扑感知的GPU分配
- 同一TP组必须在同一NVLink域

工程实践 Tips

Checkpoint策略:

- 异步checkpoint (不阻塞训练)
- 分布式存储 (每GPU写自己的shard)
- 增量checkpoint减少I/O

故障检测:

- NCCL watchdog: 通信超时检测
- 心跳机制: 每30s检查所有rank
- NaN检测: loss异常立即报警

LLaMA-3训练报告:

- 54天训练, 419次中断
- 有效训练时间占比: ~90%
- 主要故障: GPU ECC错误, NCCL超时
- 自动恢复平均耗时: 8.3分钟

Key Takeaway: 千卡训练的工程瓶颈从"算法"转向"可靠性": checkpoint, 弹性训练, 拓扑感知调度缺一不可

PART 09

自动并行：让系统自己找最优策略

Automatic Parallelism: From Manual Tuning to Algorithmic Search

为什么需要自动并行?

搜索空间爆炸: 手动配置在新模型/新硬件上越来越不可行

搜索空间分析

配置维度	可选值
TP:	{1, 2, 4, 8}
PP:	{1, 2, 4, 8, 16, 32}
DP:	$N / (TP * PP)$
micro_batch:	{1, 2, 4, 8, 16, 32}
ZeRO stage:	{0, 1, 2, 3}
recompute:	{none, selective, full}

组合数: $4*6*6*4*3 = 1,728$ (仅单层)

多层异构: 每层不同 $\rightarrow 1728^L$

手动调优的痛点

- 需要深度硬件知识 (NVLink拓扑, IB带宽)
- 每换一次模型架构需重新调
- 每换一次集群需重新调
- GPT-3级别: 专家团队数周
- 新模型(MoE, 长seq)让搜索空间更大

$\rightarrow$ 能否让算法自动搜索最优策略?

自动并行的目标: 给定模型M + 集群C + 约束(内存, 延迟), 自动找到最大吞吐的并行策略

$\min T(\text{config}) \quad \text{s.t.} \quad \text{mem}(\text{config}) \leq \text{GPU_mem}, \text{ config in search_space}$

Key Takeaway: 手动调优需要专家知识且不可迁移, 搜索空间随模型复杂度指数增长

Alpa: 两层自动并行 (OSDI'22)

核心思想: 将搜索分为Inter-op (层间) + Intra-op (层内) 两层, 分而治之

Alpa两层搜索架构

Layer 1: Inter-operator (层间)

- 决定: 哪些层分到哪个stage (PP划分)
- 算法: Dynamic Programming
- 目标: 最小化stage间延迟 + 负载均衡

v 调用 v

Layer 2: Intra-operator (层内)

- 决定: 每个tensor如何sharding (TP/DP)
- 算法: Integer Linear Programming (ILP)
- 输入: 算子图 + device mesh拓扑
- 输出: 每个tensor的最优sharding spec

关键结果

性能对比 (GPT-like models):

- 64卡: 达到手调Megatron的 96% 吞吐
- 128卡: 超越Megatron (更优的stage划分)
- 异构模型: 自动处理Encoder-Decoder

搜索时间:

- ILP求解: ~分钟级 (单次intra-op)
- DP搜索: ~小时级 (需枚举stage划分)
- 总计: 数小时 (一次性cost)

核心贡献:

证明"分层搜索"可以逼近全局最优,
同时将搜索复杂度从指数降到多项式

Zheng, Lianmin, et al. "Alpa: Automating inter-and {Intra-Operator} parallelism for distributed deep learning." 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). 2022.

Key Takeaway: Alpa将NP-hard的全局搜索分解为DP(层间)+ILP(层内), 达到手调96%+的性能

Alpa: 使用方式与内部机制

用户只需一行装饰器, 系统自动完成分析+搜索+编译+执行

```
# Alpa: 自动并行 (基于JAX)
import alpa

# 用户代码: 只需加一个装饰器
@alpa.parallelize(method=alpa.ShardParallel())
def train_step(state, batch):
    def loss_fn(params):
        logits = model.apply(params, batch["x"])
        return cross_entropy(logits, batch["y"])
    grads = jax.grad(loss_fn)(state.params)
    return state.apply_gradients(grads=grads)

# Alpa内部流程:
# 1. Trace JAX computation graph
# 2. Inter-op: DP枚举stage划分
#    -> 每种划分调用Intra-op solver
# 3. Intra-op: ILP求解最优sharding
#    -> min(通信cost) s.t. 内存约束
```

- ShardParallel: 自动混合TP+DP- PipedShardParallel:
加上PP- - ILP目标函数:- $\min \text{sum}(\text{resharding_cost})$ -
s.t. $\text{memory} \leq \text{budget}$ - sharding合法性约束- -
核心优势:- - 无需指定TP/PP/DP度数- - 自动处理异构
层- - 支持任意JAX模型- - 局限:- - 仅支持JAX (非
PyTorch)- - 搜索时间随模型增大

Key Takeaway: Alpa将并行策略搜索完全自动化, 用户无需理解TP/PP/DP的细节

更多自动并行: Galvatron & FlexFlow

不同搜索算法的权衡: 搜索质量 vs 搜索时间 vs 搜索空间覆盖

方法	搜索算法	搜索空间	搜索时间	特点
Alpa (OSDI'22)	DP + ILP	TP/DP sharding + PP stages	小时级	最优性保证 (在分层假设下)
Galvatron (VLDB'23)	Decision Tree + cost model	TP/DP/PP/SP 组合搜索	分钟级	快速, 支持 PyTorch
FlexFlow (MLSys'19)	MCMC over SOAP space	任意算子 级别sharding	小时级	最细粒度, 通用性强
Unity (OSDI'22)	MCMC + cost model	算子+并行 联合优化	小时级	算子融合+ 并行联合搜索

Galvatron (VLDB'23) 亮点:

- 基于决策树的层次化搜索: 先确定TP -> 再确定PP -> 最后DP
- 搜索时间: 分钟级 (vs Alpa小时级), 代价是可能错过非层次化最优
- 原生PyTorch支持, 与Megatron-LM兼容, 实用性更强
- 性能: 128 GPU上与Megatron-LM持平或更优 (GPT/T5/Swin)

Key Takeaway: 自动并行三大流派: ILP精确求解(Alpa), 决策树快速搜索(Galvatron), MCMC随机搜索(FlexFlow)

自动并行: 现状与展望

理想很丰满: 大多数生产系统仍依赖手动配置, 原因何在?

与生产的差距

1. Cost Model精度

- 实际通信延迟受拥塞/调度影响
- 静态模型误差10-30%
- Profile-based更准但更慢

2. 搜索速度

- 大模型(>100B)搜索空间爆炸
- 每次硬件变化需重新搜索

3. 算子覆盖

- 自定义算子难以自动分析
- Flash Attention等特殊实现

4. 异构硬件

- 混合GPU型号 (A100+H100)
- 非均匀网络拓扑

未来方向

趋势1: Compiler-integrated

- XLA/Triton集成并行搜索
- GSPMD (Google) 已在TPU生产

趋势2: LLM-assisted

- 用LLM预测最优配置
- Few-shot从历史配置学习

趋势3: Online adaptation

- 训练中动态调整策略
- 根据实际通信profile优化

当前现实:

Meta/Google/NVIDIA: 仍手动调
Alpa/Galvatron: 学术验证阶段
GSPMD: 最接近生产的自动方案

Key Takeaway: 自动并行仍是开放问题: cost model精度、搜索速度、算子覆盖是三大瓶颈

PART 10

并行推理：部署阶段的新战场

Parallel Inference: From Training Throughput to Serving Latency

训练并行 vs 推理并行

Ch4讲了prefill/decode区别, 这里聚焦: 推理需要怎样不同的并行策略?

特性	训练 (Training)	推理 (Inference)
目标	最大化吞吐量 (tokens/sec)	最小化延迟 + 满足SLO
计算模式	前向 + 反向, 大batch	前向, 自回归逐token
内存占用	参数 + 梯度 + 优化器 + 激活值	参数 + KV cache (随seq增长)
Batch特点	固定大batch, 同步	动态batch, 异步请求流
并行需求	TP+PP+DP (吞吐优先)	TP为主 (延迟优先), PP谨慎用

推理并行的核心挑战:

1. Decode阶段: 每次只生成1 token, 计算量小, 但权重重要全部加载 -> memory-bound
2. PP在推理中引入额外延迟: 无法用微批次掩盖bubble (请求是逐token的)
3. KV cache随序列增长, 需要跨GPU分布 -> TP自然分摊KV cache

Key Takeaway: 推理并行从"吞吐优先"转向"延迟优先", TP是主力, PP需谨慎, 新策略(投机解码/解聚)应运而生

推理中的Tensor Parallelism

TP是推理并行的首选: 同时切分权重和KV cache, 降低延迟

TP推理的双重收益

收益1: 内存 -- 权重分摊

LLaMA-70B (FP16): 140GB -> TP=8: 18GB/卡

收益2: 延迟 -- 并行GEMM

Decode: 每卡GEMM小8x -> AllReduce合并

总延迟 < 单卡 (当AR overhead < 计算节省)

收益3: KV cache自动分摊

每卡只存 num_heads/TP 个head的KV cache

TensorRT-LLM TP配置

```
# TensorRT-LLM: TP推理部署
from tensorrt_llm import Mapping

mapping = Mapping(
    world_size=8,
    tp_size=8,    # 8-way tensor parallel
    pp_size=1,    # 推理中通常不用PP
)

# 构建引擎
build_config = BuildConfig(
    max_batch_size=64,
```

TP推理的扩展性极限:

- TP=2: 延迟降低~40% (2x GEMM加速, AR开销小) - TP=4: 延迟降低~65%
- TP=8: 延迟降低~75% (AR开销占比增大) - TP>8: 收益递减, 需NVLink (跨节点TP不划算)
- 经验法则: 推理TP度数 = $\min(\text{NVLink GPU数}, \text{ceil}(\text{model_size} / \text{GPU_mem}))$

Key Takeaway: 推理TP同时分摊权重、计算和KV cache, 但受NVLink拓扑限制, 通常TP<=8

投机解码 (Speculative Decoding)

核心思想: 用小模型猜, 大模型一次验证, 以计算换延迟

工作原理

Step 1: Draft Model (7B) 自回归生成k=5个token

Step 2: Target Model (70B) 对所有k个token做一次前向

Step 3: 逐位置比较 draft vs target 的概率分布

Step 4: Accept/Reject (保证输出分布 = target分布)

为什么有效: 大模型decode是memory-bound, 一次验5个token和验1个耗时几乎相同!

理论分析

接受率分析:

$$P(\text{accept token } i) = \min(1, q(x)/p(x))$$

q: draft分布, p: target分布

期望接受长度:

$$E[\text{accepted}] = \sum(\alpha^i, i=0..k-1)$$

alpha = avg acceptance rate

加速比:

实测 (LLaMA-70B + LLaMA-7B draft):

- Code: alpha~0.8, 2.5-3x speedup
- Chat: alpha~0.6, 1.8-2.2x speedup
- 保证: 输出分布与target完全一致!

Key Takeaway: 投机解码: 小模型猜k个token, 大模型一次验证, 无损加速2-3x (memory-bound场景)

投机解码: 代码与变体

从标准投机解码到Medusa/Eagle: 去掉draft model, 更快更简单

投机解码伪代码

```
def speculative_decode(prompt, draft_model, target_model, k=5):  
    output = []  
    while not done:  
        # Step 1: Draft model快速生成k个候选  
        drafts, draft_probs = draft_model.generate(  
            prompt + output, num_tokens=k  
        )  
        # Step 2: Target model一次前向验证全部  
        target_probs = target_model.forward(  
            prompt + output + drafts # batch化  
        )  
        # Step 3: Accept/reject (保持分布一致)  
        for i in range(k):  
            r = random.uniform(0, 1)  
            if r < min(1, target_probs[i] / draft_probs[i]):  
                output.append(drafts[i]) # 接受  
            else:
```

- 标准投机解码需要额外draft模型

- 变体方法 (去掉draft model):-

Medusa (2024): 给target加多个预测head, 并行预测k个位置, 无需额外模型,

tree-verify- - Eagle (2024): 轻量特征外推网络, 利用target隐层特征, 接受率比Medusa高10-20%。

Key Takeaway: 投机解码是推理加速的重要方向, Medusa/Eagle通过去掉draft model简化部署

解聚推理 (Disaggregated Inference)

核心洞察: Prefill(compute-bound) 和 Decode(memory-bound) 的硬件需求完全不同

Prefill 阶段

- 一次处理整个prompt (数千token)
- Compute-bound: 大矩阵乘法, GPU利用率高
- 需要: 高FLOPS -> 高端GPU, 大batch
- 适合: 密集计算型GPU (H100 SXM)

Decode 阶段

- 逐token生成, 每次只算1个token
- Memory-bound: 加载全部权重只做小GEMV
- 需要: 高带宽 -> HBM带宽, 大batch聚合
- 适合: 多卡分摊权重, 或低功耗加速器

解聚架构

传统: Prefill + Decode 在同一GPU
-> Decode被Prefill抢占, TTFT波动大

解聚: 分离到不同GPU池

[请求] -> [Prefill Pool]
| KV cache传输
[Decode Pool] -> [输出]

关键挑战: KV cache传输

- LLaMA-70B, seq=4K:
KV cache = $2 \times 70 \times 2 \times 4096 \times 128 \times 2B$
= ~4.5GB per request
- 需要高速网络 (RDMA/NVLink)
- KV cache压缩可减少50-70%传输量

Key Takeaway: 解聚推理将Prefill和Decode分离部署, 各自优化硬件配置, 突破混合部署的资源浪费

Splitwise & DistServe: 解聚推理实践

两篇顶会论文: 从理论到系统, 解聚推理的工程落地

Splitwise (ISCA'24)

核心贡献:

- 首次系统性分析PD解聚的收益
- 在相同SLO下, 吞吐提升1.4x

关键发现:

- Prefill GPU利用率: 60-80%
- Decode GPU利用率: 仅5-15%
- 混合部署: 互相干扰, 两者都差

解聚收益来源:

1. Prefill池可独立扩batch -> 高利用率
2. Decode池聚合多请求 -> 提升MBU
3. 各池独立scaling -> 弹性伸缩

DistServe (OSDI'24)

核心贡献:

- Goodput-optimal: 最大化满足SLO的请求数
- 为P/D分别搜索最优并行策略

关键设计:

Prefill: TP=4, 大batch, 高FLOPS

Decode: TP=2, 更多实例, 高并发

KV cache传输优化:

- Pipeline KV transfer with compute
- 分层传输: 生成一层传一层
- 传输延迟被计算掩盖

**结果: vs vLLM (混合), goodput提升
2-5x at strict SLO targets**

Key Takeaway: Splitwise证明解聚的收益(1.4x), DistServe给出最优部署策略(goodput 2-5x), KV传输是关键

分布式vLLM: 多GPU推理引擎

Ch3/4预览过vLLM核心, 这里聚焦分布式部署: TP + continuous batching + PagedAttention

vLLM分布式架构

Scheduler (中心调度器)

- 管理请求队列, 决定batch组成
- Continuous batching: 动态加入/移除请求

TP Worker Group (TP=8)

- 每个worker持有权重shard
- NCCL AllReduce同步激活值

PagedAttention KV Manager

- KV cache按page分配 (类虚拟内存)
- 跨TP worker的page table同步

vLLM分布式部署

vLLM多GPU部署

单节点多卡 (TP)

```
python -m vllm.entrypoints.api_server \
    --model meta-llama/Llama-3-70B \
    --tensor-parallel-size 8 \
    --max-num-seqs 256 \
    --gpu-memory-utilization 0.9
```

多节点部署 (TP跨节点 or PP)

Node 0 (head):

ray start --head

```
python -m vllm.entrypoints.api_server \
    --model meta-llama/Llama-3-405B \
    --tensor-parallel-size 8 \
    --pipeline-parallel-size 4 \
    --distributed-executor ray
```

关键参数:

--max-num-seqs: 最大并发请求数

--block-size: KV cache page大小

--swap-space: CPU swap空间(GB)

Key Takeaway: vLLM = TP权重分摊 + PagedAttention内存管理 + Continuous Batching动态调度, 开箱即用

推理并行前沿: 未来方向

推理系统正在从"单一优化"走向"全栈协同": 解聚 + TP + 压缩 + 调度

PD解聚 + TP + PP 组合

- Prefill: TP=4 (高计算密度)
- Decode: TP=2 + PP=2 (高并发)
- 各阶段独立弹性伸缩
- Mooncake (2024): 字节跳动生产

KV Cache压缩与共享

- 量化: FP16->INT4 (4x压缩)
- Pruning: 丢弃不重要的KV对
- Cross-request共享 (SGLang RadixAttention)
- 大幅减少传输和存储开销

Prefill调度优化

- Chunked prefill: 长prompt分块处理
- 与decode交错执行, 减少TTFT
- Sarathi-Serve: 自适应chunk大小
- 平衡TTFT与TPOT的SLO

Prefix Caching

- 系统prompt的KV cache复用
- vLLM APC: 自动前缀缓存
- SGLang: Radix tree管理prefix
- 相同prefix的请求共享计算

Key Takeaway: 推理优化正从单点技术走向全栈: 解聚+压缩+调度+缓存的协同优化是趋势

训练与推理框架全景

选择框架: 训练看扩展性和并行策略支持, 推理看延迟和吞吐

训练框架

框架	TP	PP	ZeRO	EP	适用场景
Megatron-LM	Y	Y	-	Y	大规模预训练
DeepSpeed	Y	Y	0-3	Y	通用, ZeRO系列
FSDP	-	-	~3	-	PyTorch原生

推理框架

框架	TP	Batching	PagedAttn	Spec Dec	适用场景
vLLM	Y	Continuous	Y	Y	通用, 开源首选
TensorRT-LLM	Y	In-flight	Y	Y	NVIDIA最优性能
SGLang	Y	Continuous	Y	Y	结构化生成, RadixAttn

趋势: 训练框架趋向Megatron-DeepSpeed融合; 推理框架趋向vLLM+SGLang的开源生态 vs TensorRT-LLM的性能极致

Key Takeaway: 训练: Megatron-LM(大规模) + DeepSpeed(通用); 推理: vLLM(开源) + TensorRT-LLM(性能)

第六章全景: 从单卡到千卡的完整路线

知识地图: 每个节点代表一个核心概念, 箭头表示演化关系



Key Takeaway: 完整路线: 单卡 -> DP/DDP/ZeRO -> TP/PP/SP -> 3D混合 -> 自动并行 -> 并行推理

课堂讨论: 设计与分析题

将本章知识应用于实际场景: 三道开放性设计问题

问题1: 并行策略设计

给定 LLaMA-70B (FP16, 140GB) 和 32 x H100 (4节点, 每节点8卡NVLink).

- 设计并行策略 (TP, PP, DP)
- 计算每卡内存占用 (参数 + KV cache)
- 如果要支持128K context, 策略如何调整?

问题2: 通信瓶颈分析

配置: TP=8, PP=4, DP=8, 集群256 x H100.

- 哪个维度是通信瓶颈? 定量分析
- 如果将TP从NVLink移到IB, 性能会下降多少?
- 如何用ZeRO/FSDP优化DP维度的通信?

问题3: 推理策略对比

对比投机解码 vs 解聚推理:

- 分别适用于什么场景? (延迟 vs 吞吐)
- 能否结合使用? 如何设计组合方案?
- 如果请求模式为: 长prompt + 短回复, 哪种更优?
反过来呢 (短prompt + 长回复)?

Key Takeaway: 好的并行策略 = 理解硬件拓扑 + 匹配通信带宽 + 选对算法。动手算一算!